

Trabajo Final

Integrador

Detección de Anomalías en Imágenes Industriales: Análisis de Defectos en Tornillos

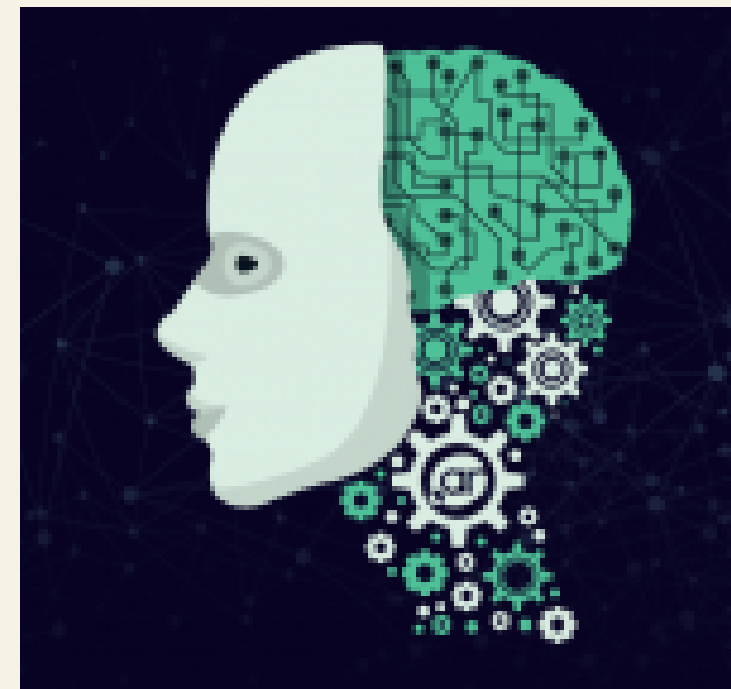
Pereyra Alexis Sebastian

Profesores:

Krujoski Matias Gabriel, Pryszczuk Sabrina Daiana y Skrauba Axel Alfredo

Contexto

- La detección de anomalías en entornos industriales es clave para garantizar el control de calidad y optimizar procesos de manufactura.
- Los sistemas tradicionales dependen de operadores humanos lo cual generan variabilidad, fatiga y limitaciones de velocidad
- Machine Learning y Deep Learning ofrecen una alternativa robusta, capaz de identificar patrones sutiles que escapan al ojo humano.



Caso de estudio: tornillos industriales del dataset MVTec Anomaly Detection.

Objetivos

- Desarrollar un pipeline completo para identificar y localizar anomalías en imágenes de tornillos.
- Analizar sesgos y robustez del modelo frente a variaciones típicas del entorno industrial.
- Probar enfoques clásicos y de deep learning

Objetivos específicos

- Evaluar la viabilidad del modelo para realizar inferencia sobre imágenes de tornillos en un entorno de producción real, considerando las limitaciones de los datos disponibles.
- Analizar posibles sesgos en el modelo y su impacto en las decisiones de control de calidad dentro de la línea de producción.
- Determinar la factibilidad computacional del pipeline propuesto para implementación en tiempo real en un entorno industrial.

Datos disponibles

MVTec AD (Bergmann et al., 2019)

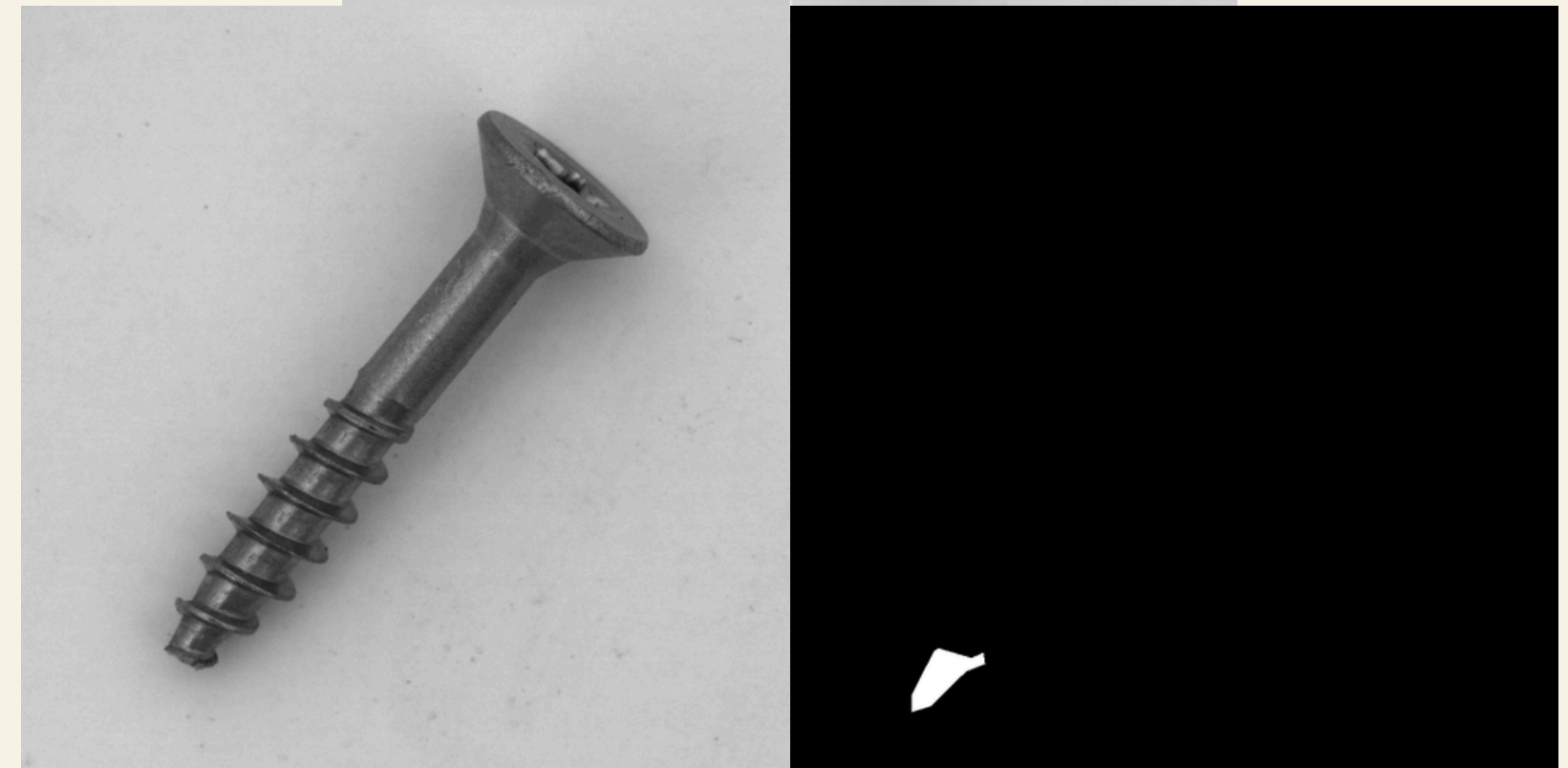
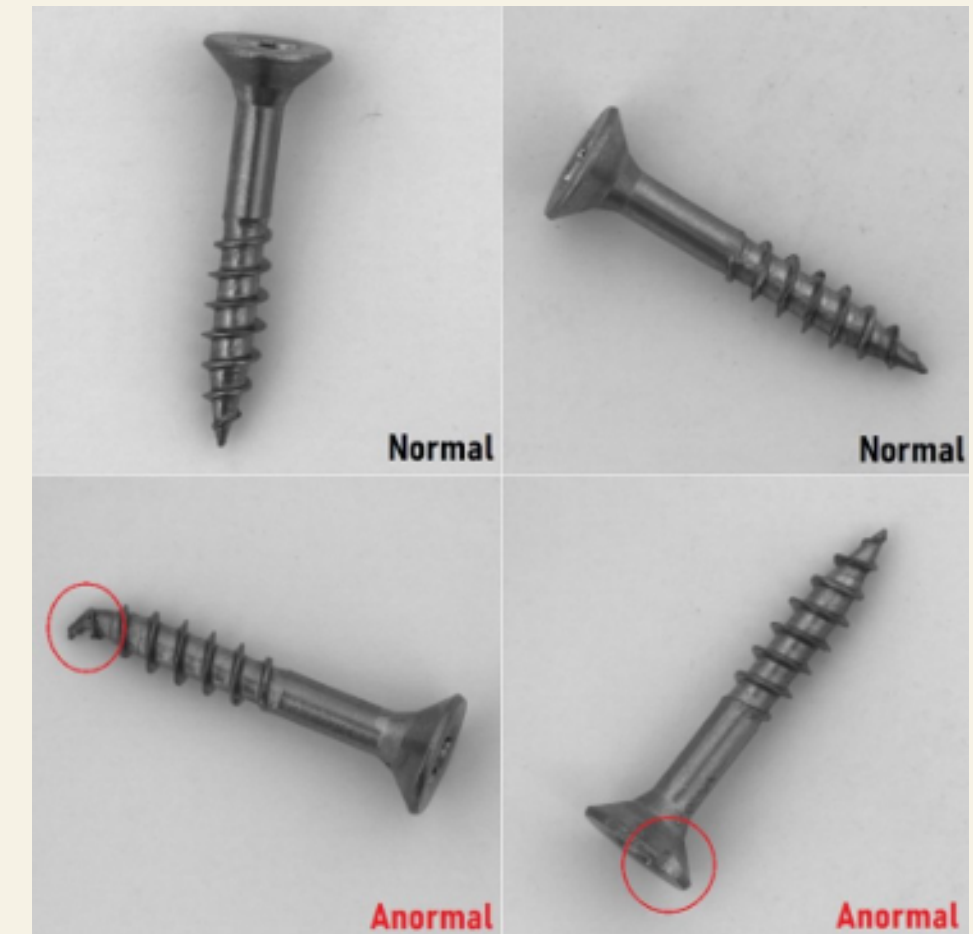
Licencia CC BY-NC-SA 4.0

Conjunto de Entrenamiento: 320 imágenes

- Conjunto de Prueba: 160 imágenes.

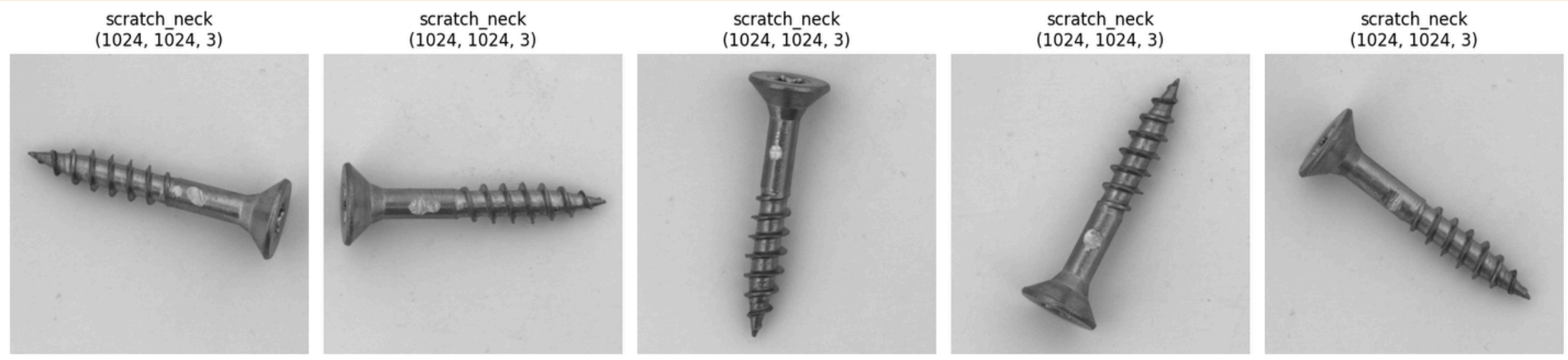
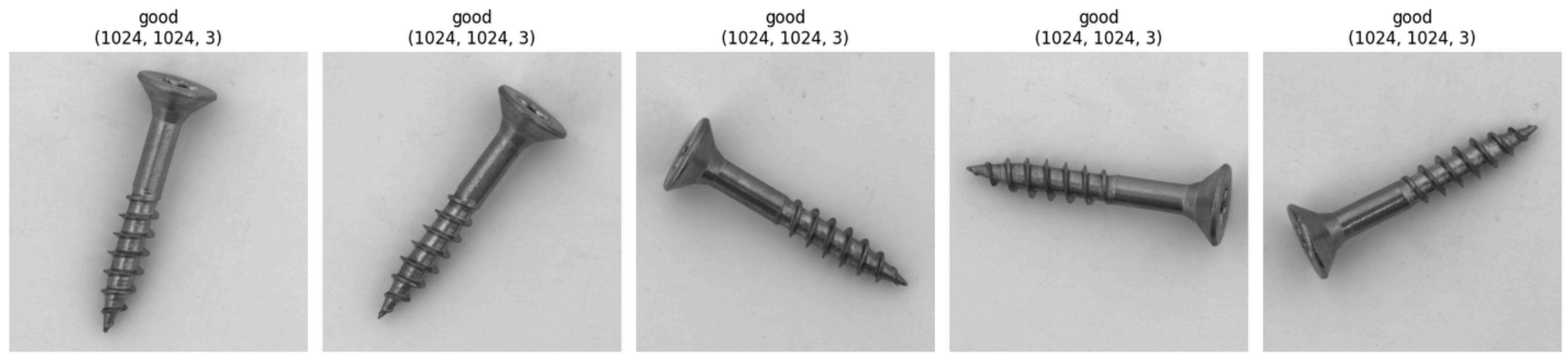
- good
- manipulated_front
- scratch_head
- scratch_neck
- thread_side
- thread_top

- Ground Truth.



Análisis de Datos

Visualización de datos



Análisis de Datos

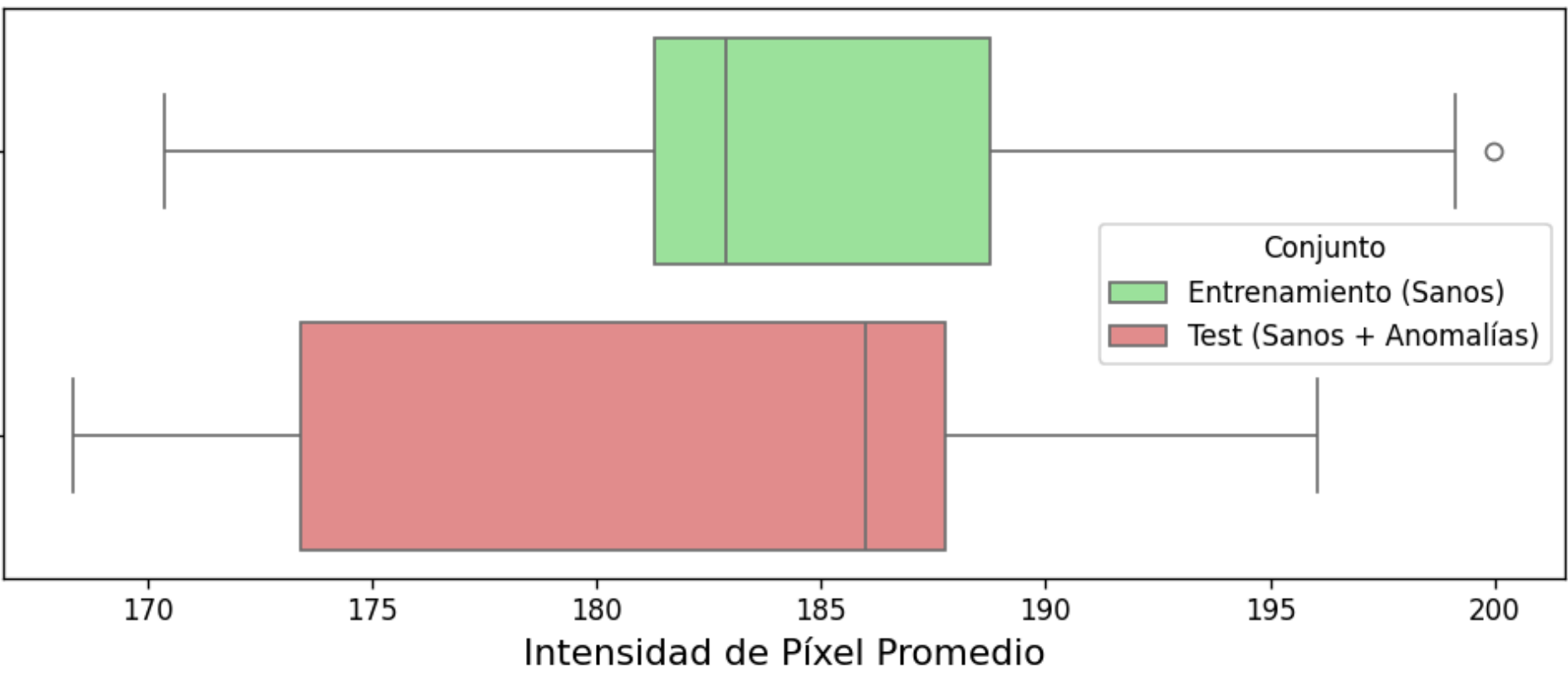
EDA

Análisis de outliers mediante PCA
N° componentes 30, con 90%



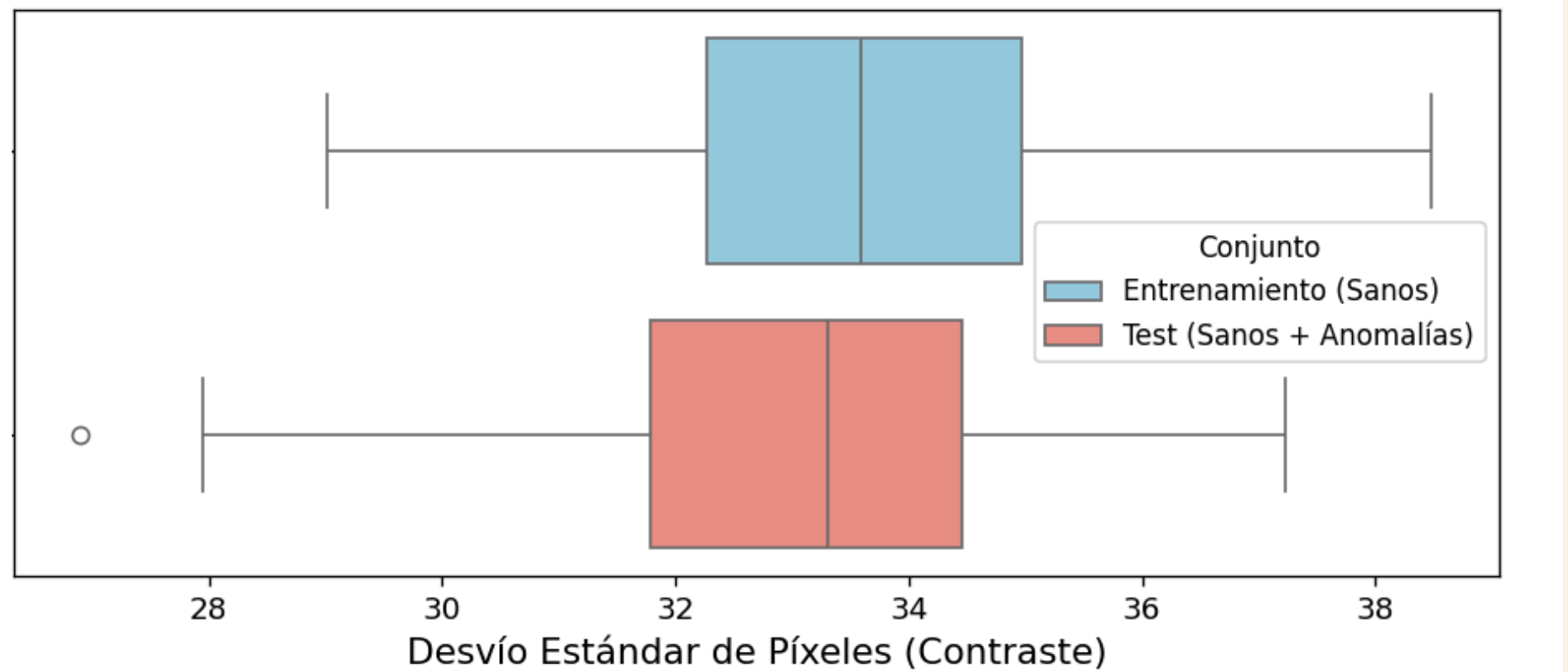
Análisis de distribución lumínica

Comparación de Distribución Lumínica: Entrenamiento vs. Test



Análisis de distribución de contraste

Comparación del Nivel de Contraste (Datos Crudos): Entrenamiento vs. Test



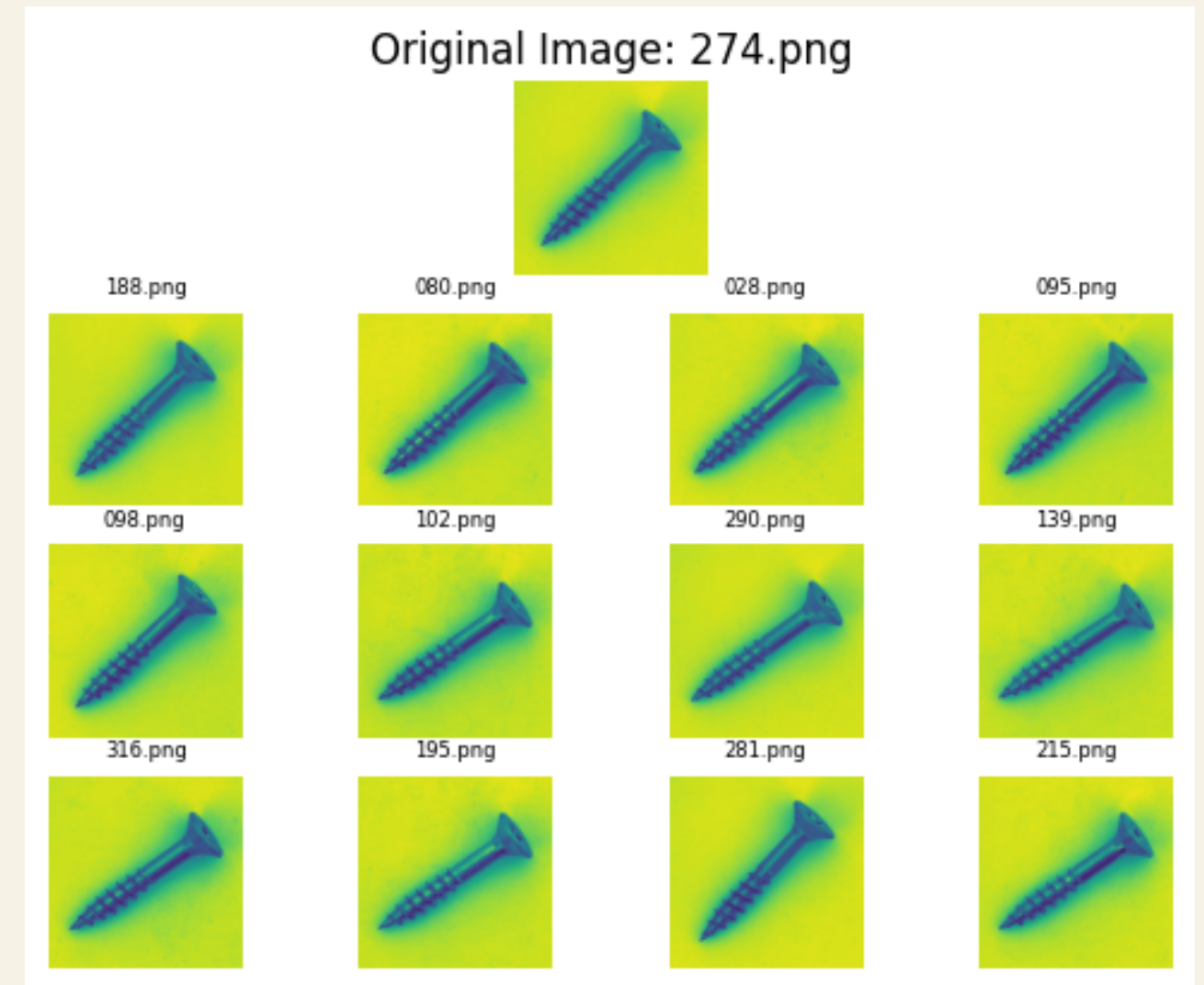
Análisis de Datos

Búsqueda de duplicados

- Phash
- Phash + SSIM
- ResNet18 + Similitud del coseno – 0.985

Dataset

- Se eliminaron 135 copias quedando 226
- Quedando 180 para el entrenamiento y 46 muestras para el test



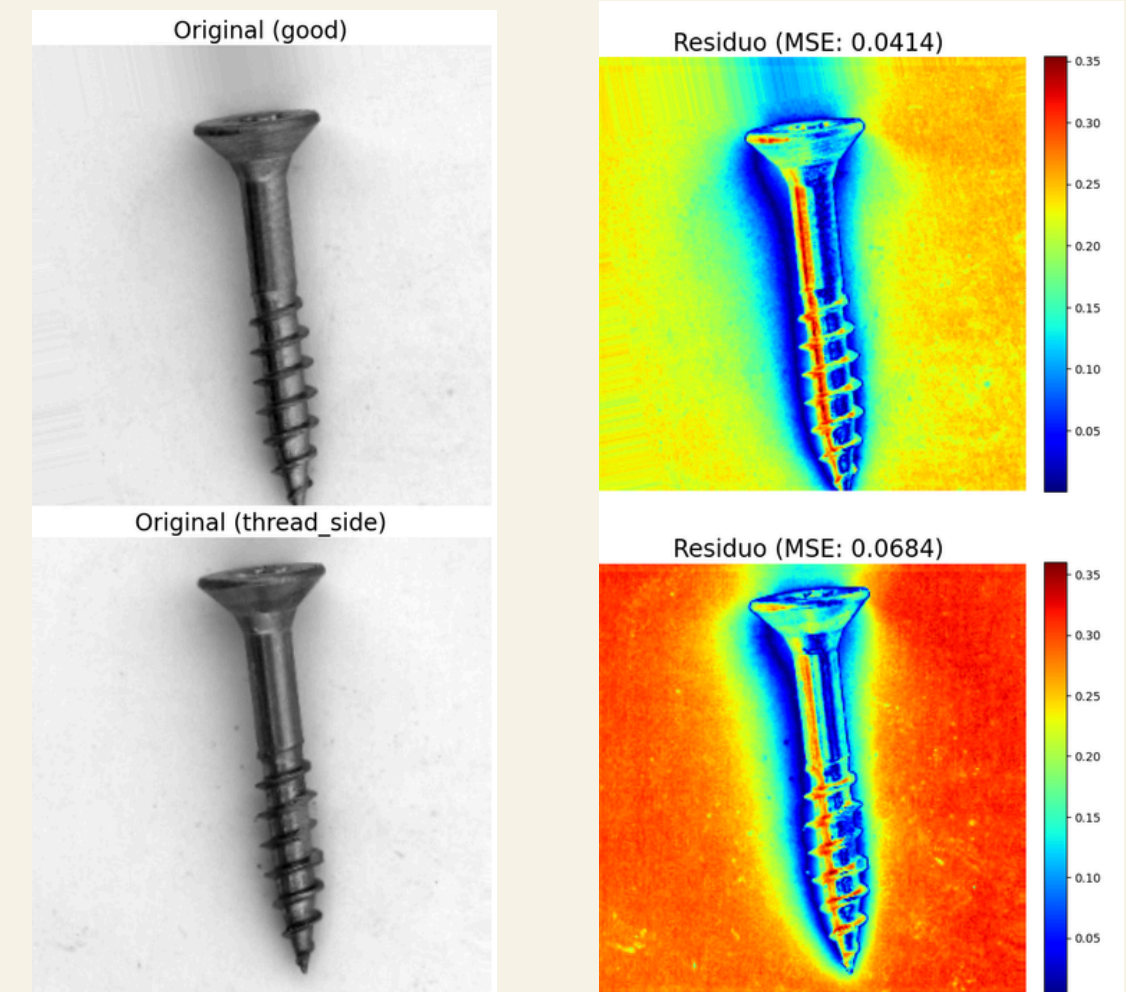
Eliminacion de fondo y alineación

Para la eliminación del fondo:

- Se escaló en grises y se mete un desenfoque leve
- Recorta el fondo mediante Otsu
- Rellena los puntos que quedan en el medio y luego se quitan los puntos que quedan flotando sueltos
- Por último encuentra el polígono más grande y se queda este

En la alineación:

- Se endereza el tornillo mediante PCA
- Luego se asegura que la cabeza termine arriba
- Por último se centra el tornillo



Blur (5,5) - El Pincel Morfológico (7, 7) - Iteraciones 2

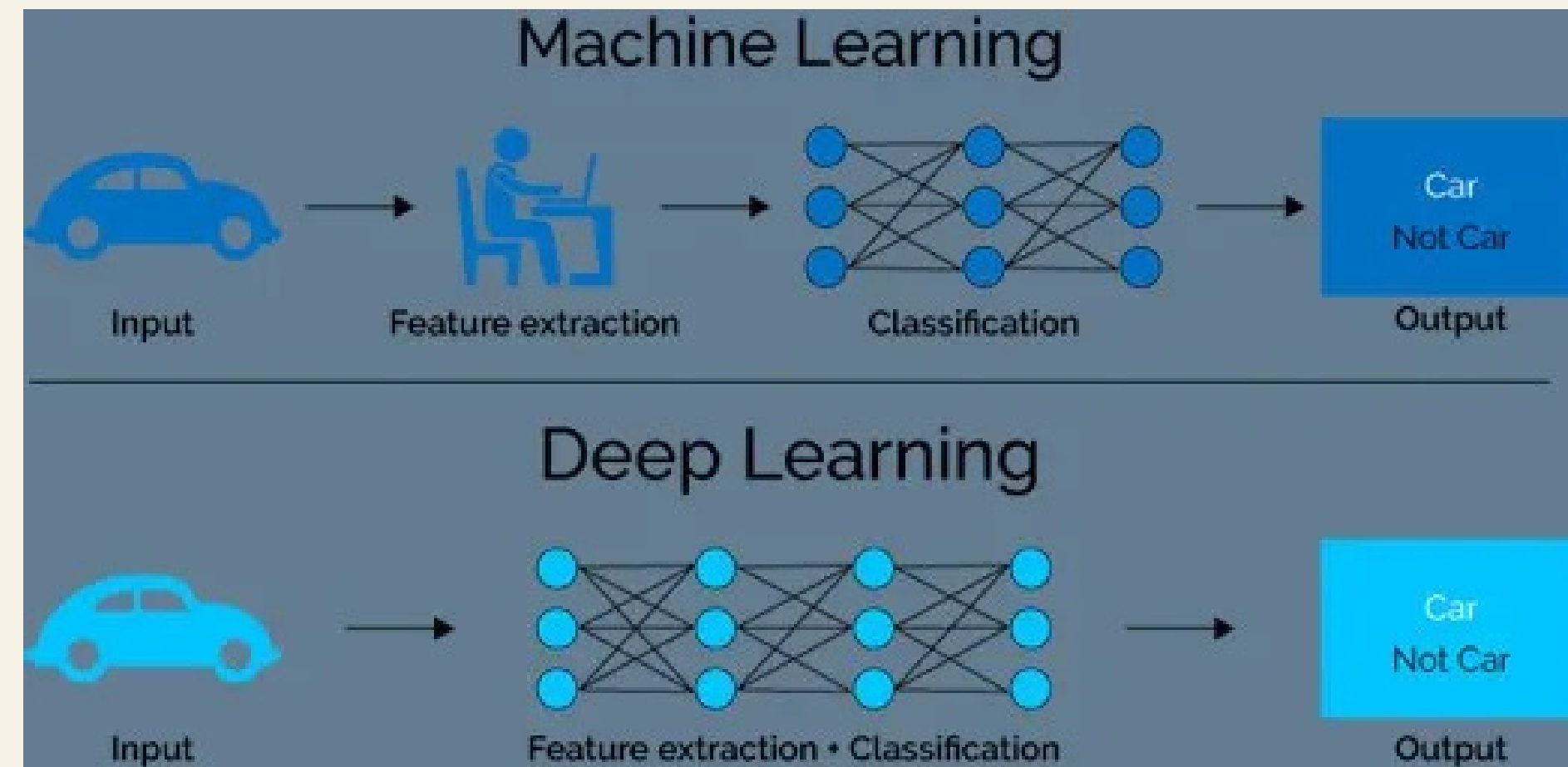
Modelos

Machine Learning

- Extracción de características + KNN
- Extracción de características + OC-SVM

Deep Learning

- Autoencoder
 - Liviano
 - Mediano
 - Pesado
- Transfer learning
 - Backbone + Decoder
 - Patchcore



Machine Learning

KNN	OC-SVM
Número de vecinos (k): [1, 3, 5, 7, 10] Métrica de distancia: euclidean, manhattan y cosine Percentil umbral: 90, 95, 97 y 99	Nu: 0.01, 0.05, 0.1 y 0.2 Kernel: rbf, linear y poly Gamma: scale y auto
Extractor de embeddings	Data Augmentation
Vit de librería transformers • Imágenes de (224, 224, 3) • google/vit-base-patch16-224-in21k Embedding → 768	Brillo y Contraste (Variaciones de $\pm 10\%$) Zoom aleatorio (Factor de 0.95 a 1.05)

Resultados

KNN

	n_neighbors	metric	percentil_umbral	tau	AUC	accuracy	precision	recall	f1_score
0	1	cosine	90	0.039018	0.557179	0.339394	0.916667	0.092437	0.167939
1	1	cosine	95	0.045714	0.557179	0.333333	0.909091	0.084034	0.153846
2	1	cosine	97	0.048733	0.557179	0.321212	0.888889	0.067227	0.125000
3	1	cosine	99	0.060005	0.557179	0.309091	0.857143	0.050420	0.095238
4	3	cosine	90	0.041518	0.556083	0.345455	0.866667	0.109244	0.194030
5	3	cosine	95	0.055522	0.556083	0.309091	0.857143	0.050420	0.095238
6	3	cosine	97	0.059123	0.556083	0.309091	0.857143	0.050420	0.095238
7	3	cosine	99	0.068141	0.556083	0.309091	0.857143	0.050420	0.095238
8	1	manhattan	99	83.469064	0.552795	0.303030	0.833333	0.042017	0.080000
9	1	manhattan	97	75.075218	0.552795	0.315152	0.875000	0.058824	0.110236

🏆 Mejor configuración encontrada:

n_neighbors=1, metric='cosine', percentil=90

AUC=0.5572 | F1=0.1679 | Precision=0.9167 | Recall=0.0924

Resultados

Knn

--- Evaluación Final (Set de Test) ---

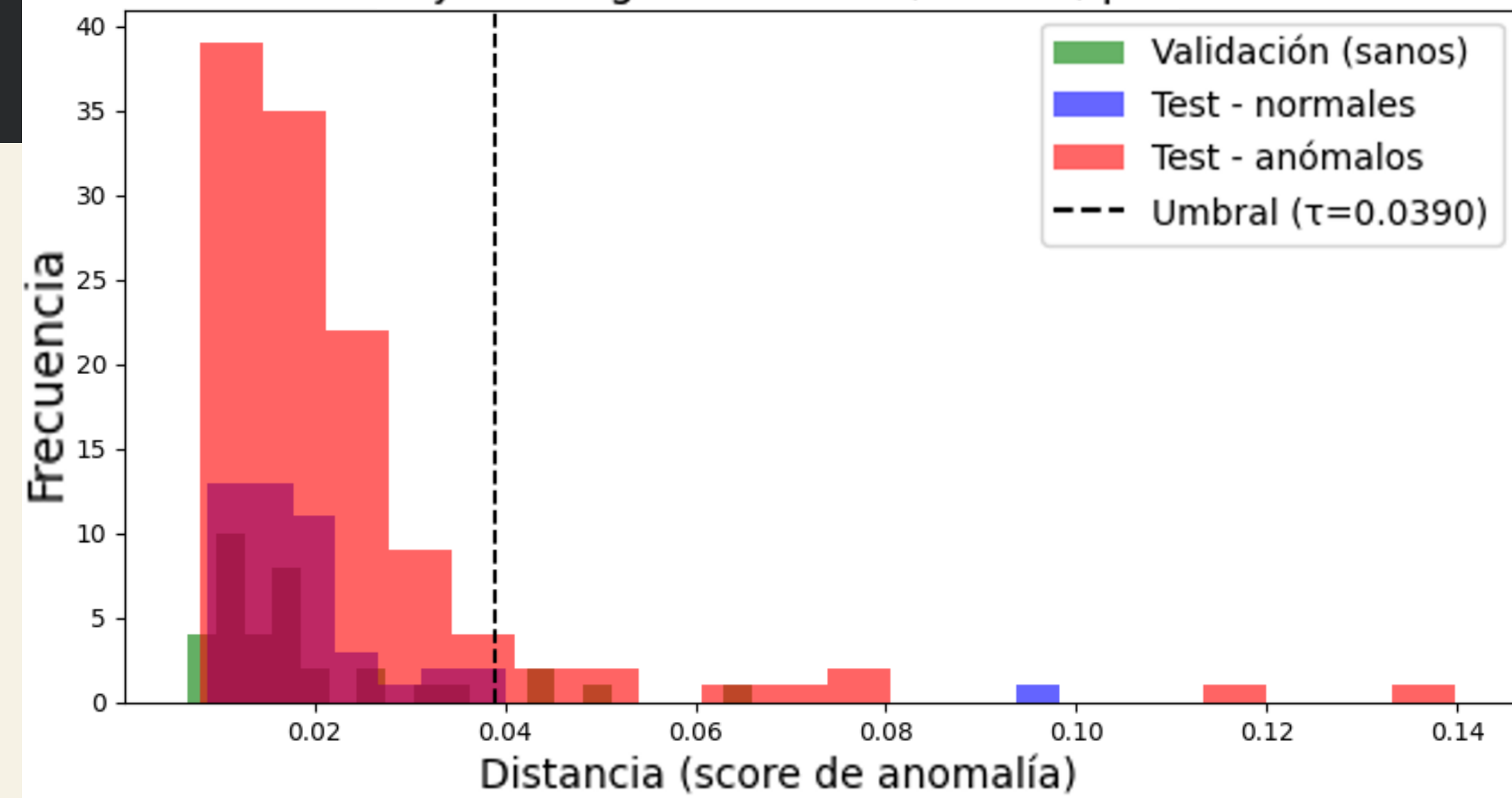
--- REPORTE FINAL DE EVALUACIÓN (Muestras totales: 165) ---

	precision	recall	f1-score	support
Sano	0.29	0.98	0.45	46
Anómalo	0.92	0.09	0.17	119
accuracy			0.34	165
macro avg	0.61	0.54	0.31	165
weighted avg	0.74	0.34	0.25	165

Matriz de Confusión:

```
[[ 45   1]
 [108  11]]
```

Mejor configuración: k=1, cosine, p90



Resultados

OC-SVM

	nu	kernel	gamma	FP_val (%sanos mal clasif.)	AUC	accuracy	precision	recall	f1_score
0	0.10	rbf	scale	0.444444	0.619839	0.569697	0.779070	0.563025	0.653659
1	0.10	rbf	auto	0.444444	0.619839	0.569697	0.779070	0.563025	0.653659
2	0.01	rbf	scale	0.444444	0.619474	0.569697	0.779070	0.563025	0.653659
3	0.01	rbf	auto	0.444444	0.619474	0.569697	0.779070	0.563025	0.653659
4	0.05	rbf	scale	0.444444	0.619474	0.569697	0.779070	0.563025	0.653659
5	0.05	rbf	auto	0.444444	0.619474	0.569697	0.779070	0.563025	0.653659
6	0.20	rbf	auto	0.444444	0.611984	0.575758	0.781609	0.571429	0.660194
7	0.20	rbf	scale	0.444444	0.611984	0.575758	0.781609	0.571429	0.660194
8	0.01	linear	scale	0.472222	0.569236	0.521212	0.750000	0.504202	0.603015
9	0.01	linear	auto	0.472222	0.569236	0.521212	0.750000	0.504202	0.603015

🏆 Mejor configuración encontrada:
nu=0.1, kernel='rbf', gamma='scale'
AUC=0.6198 | F1=0.6537 | Precision=0.7791 | Recall=0.5630

Resultados

OC-SVM

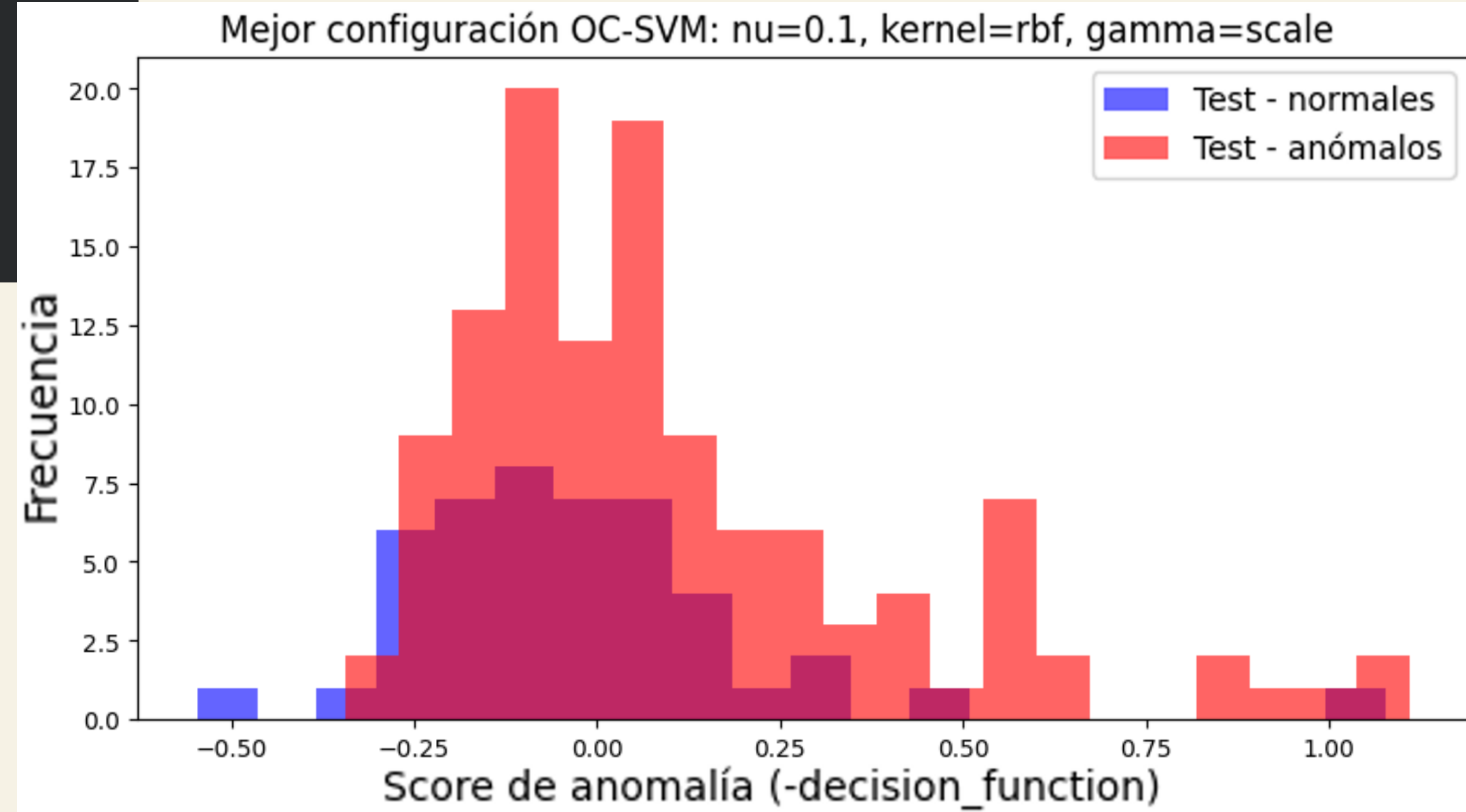
🏆 Mejor configuración encontrada:
nu=0.1, kernel='rbf', gamma='scale'
AUC=0.6198 | F1=0.6537 | Precision=0.7791 | Recall=0.5630

--- CLASSIFICATION REPORT (mejor config, sobre TEST) ---

	precision	recall	f1-score	support
Sano	0.34	0.59	0.43	46
Anómalo	0.78	0.56	0.65	119
accuracy			0.57	165
macro avg	0.56	0.57	0.54	165
weighted avg	0.66	0.57	0.59	165

--- MATRIZ DE CONFUSIÓN ---

```
[[27 19]
 [52 67]]
```



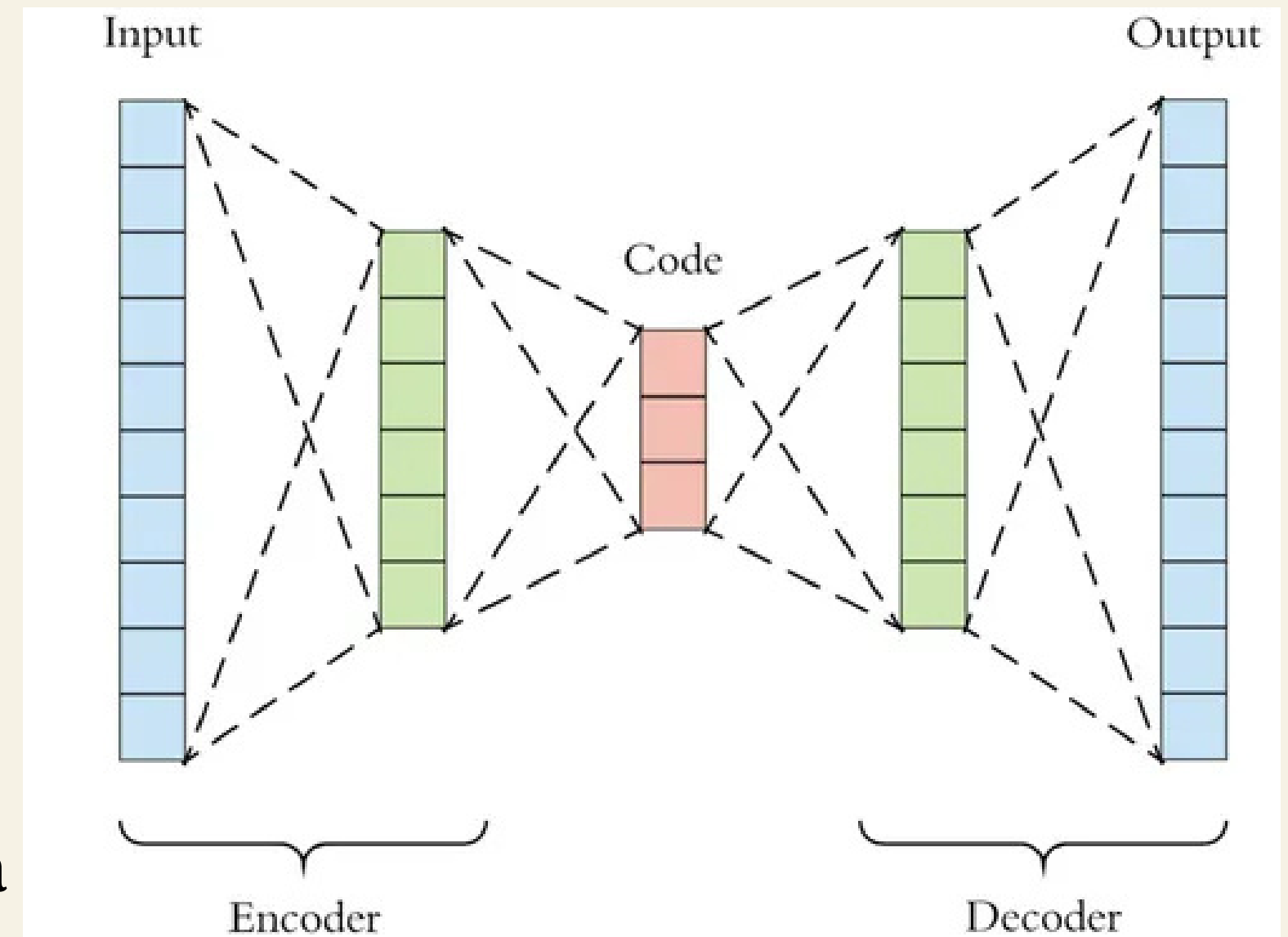
Autoencoders

Parametros del autoencoders

- Resolución de entrada 256x256x 1.
- Batch size 16
- Split del Dataset 85% para el entrenamiento y el 15% para validación.
- Optimizador: Adam, lr 5e-4

Data Augmentation

- Traslación: un 3% para arriba, abajo o hacia los costados.
- RandomZoom: hasta un 5%
- RandomRotation: 7 grados.
- RandomBrightness: Varía el brillo un 10%
- RandomContrast: el contraste un 10%.



Autoencoders

Característica	1. El Liviano	2. El Pesado	3. El Mediano
Profundidad (Escalones)	2	3	3
Capas por Bloque	1 Convolución	2 Convoluciones	1 Convolución
Cuello de Botella	64x64 espacial, 16 canales	32x32 espacial, 128 canales	32x32 espacial, 16 canales
Activación	ReLU estándar	LeakyReLU + Batch Normalization	LeakyReLU (sin BatchNorm)

CALLBACKS: Early stop (P=12, CM=1e-5), Reduce learning rate(P=5, F=0.5) y Modelcheckpoint

Autoencoders

El liviano

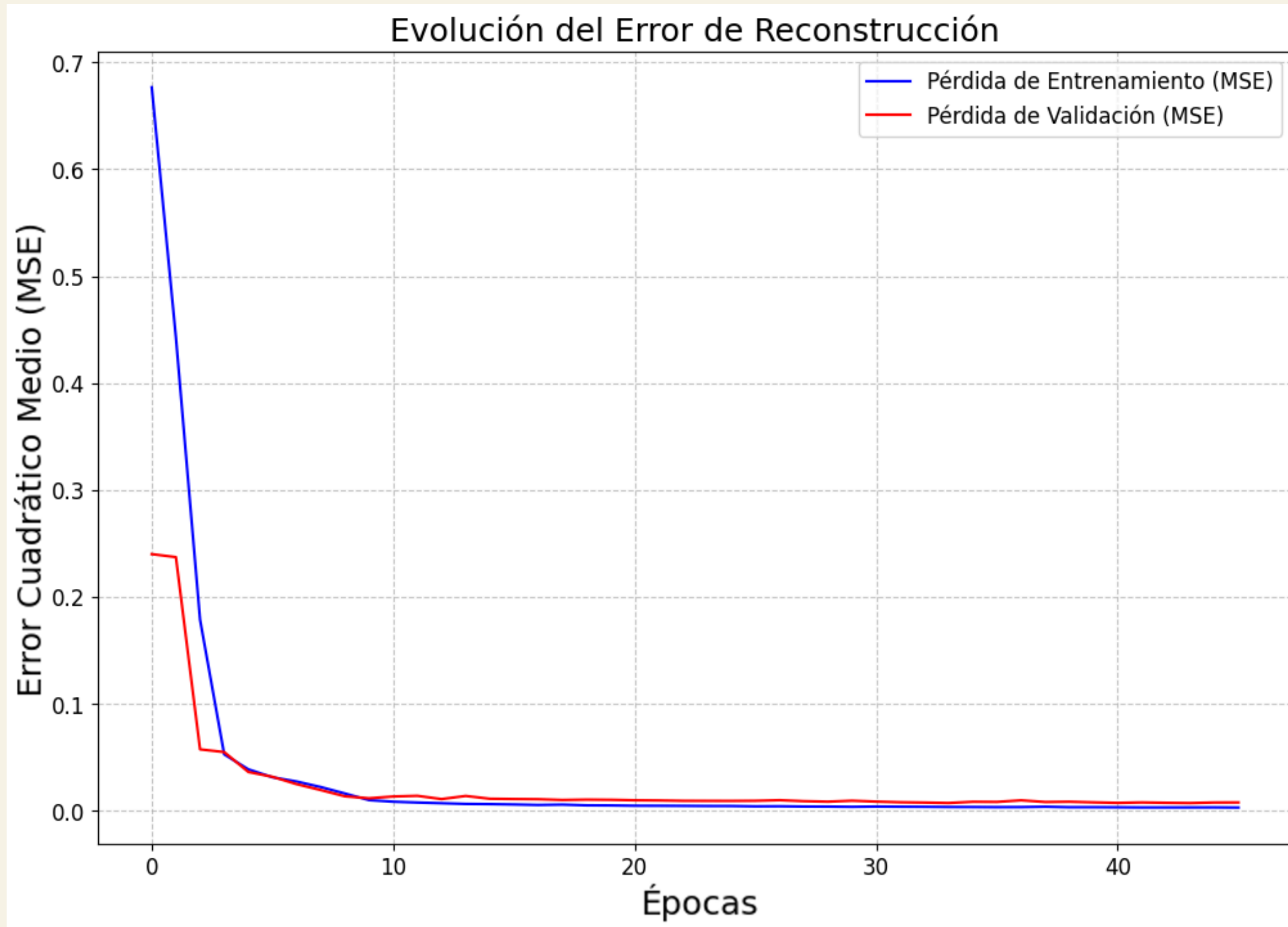


Metricas			
Versiones	Accuracy	Recall	F1-score
basico	0.28	N:1 A:0	N:0.44 A:0
DA	0.30	N:1 A:0.3	N:0.44 A:0.5
MSE-V1 + DA	0.28	N:1 A:0	N:0.44 A:0

MSE-V1: Peso del fondo 1, peso de Tornillo: 8

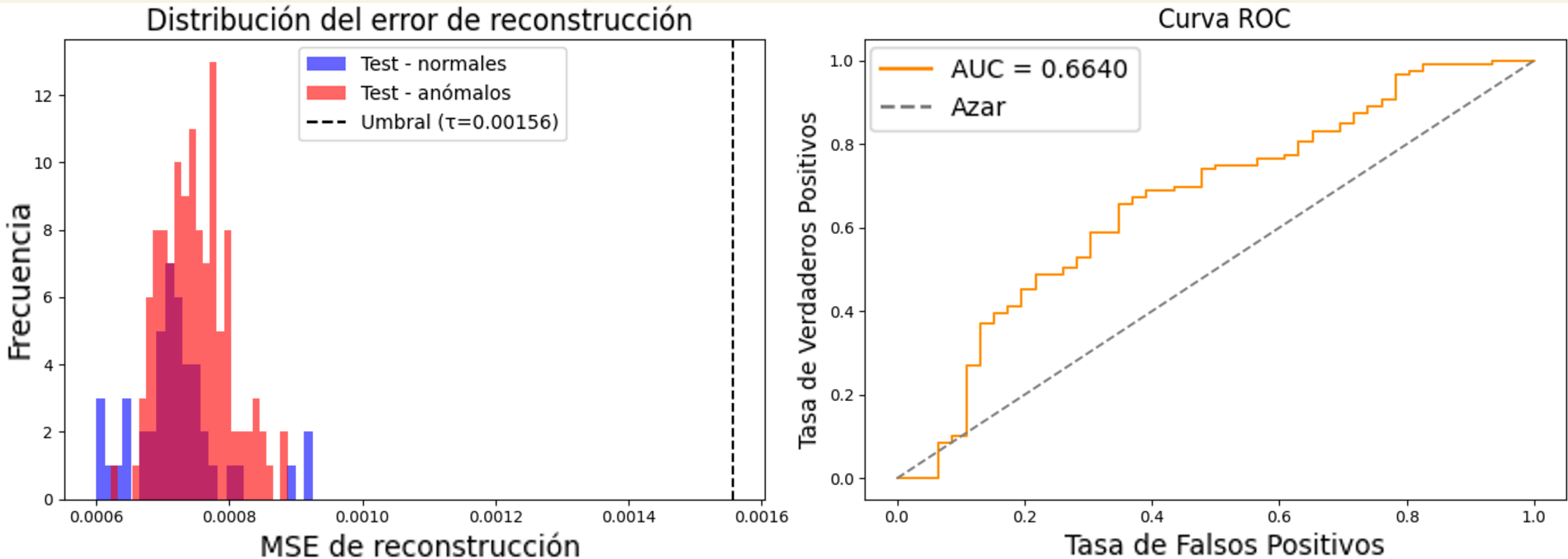
Autoencoders

MSE-V1 + DA



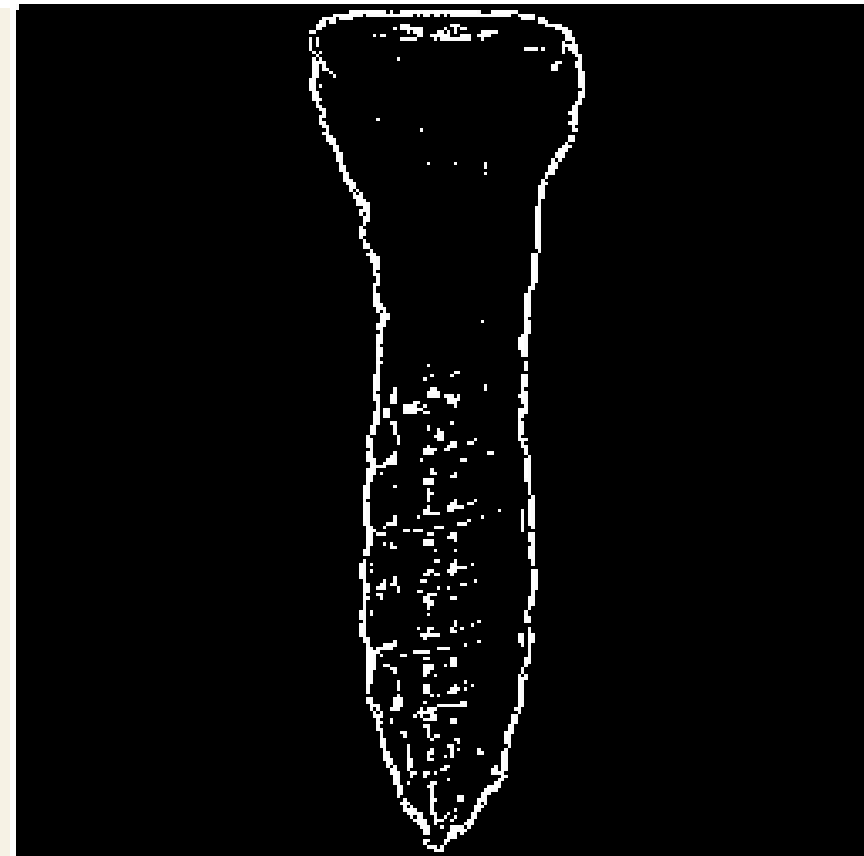
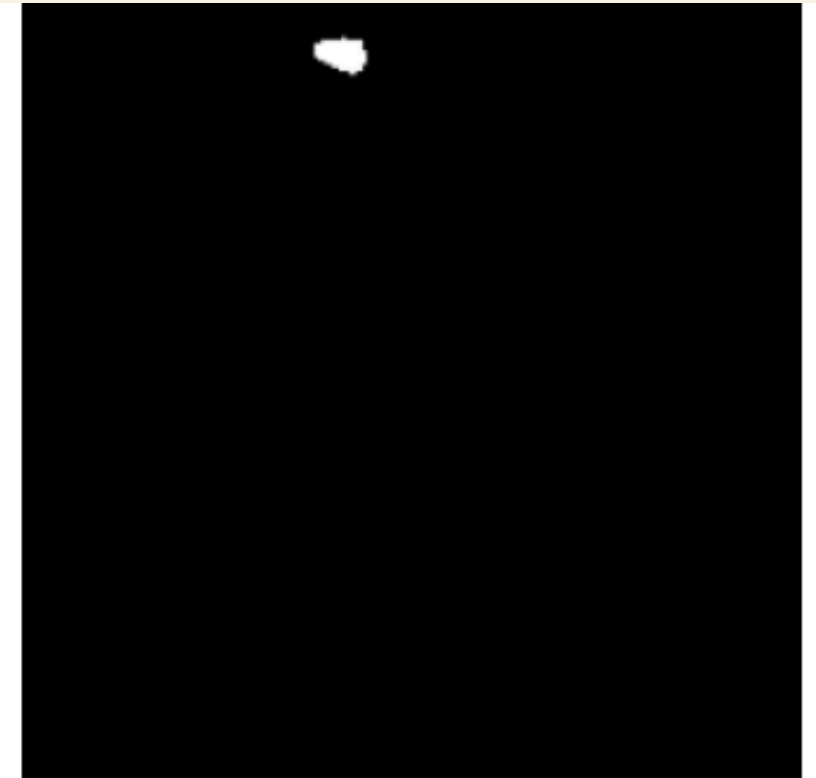
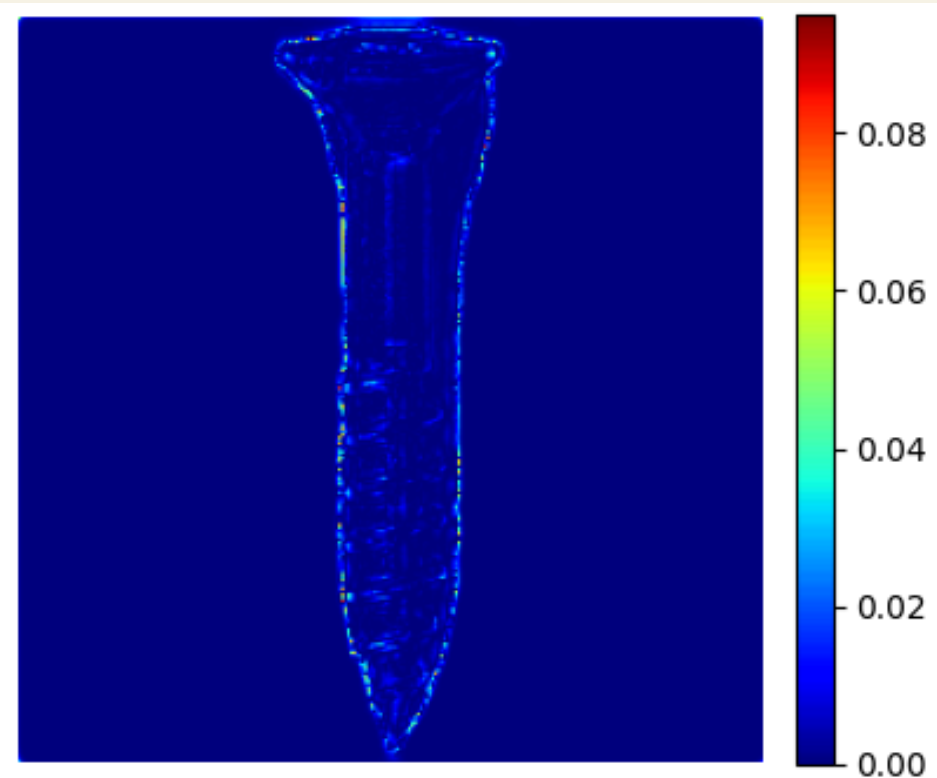
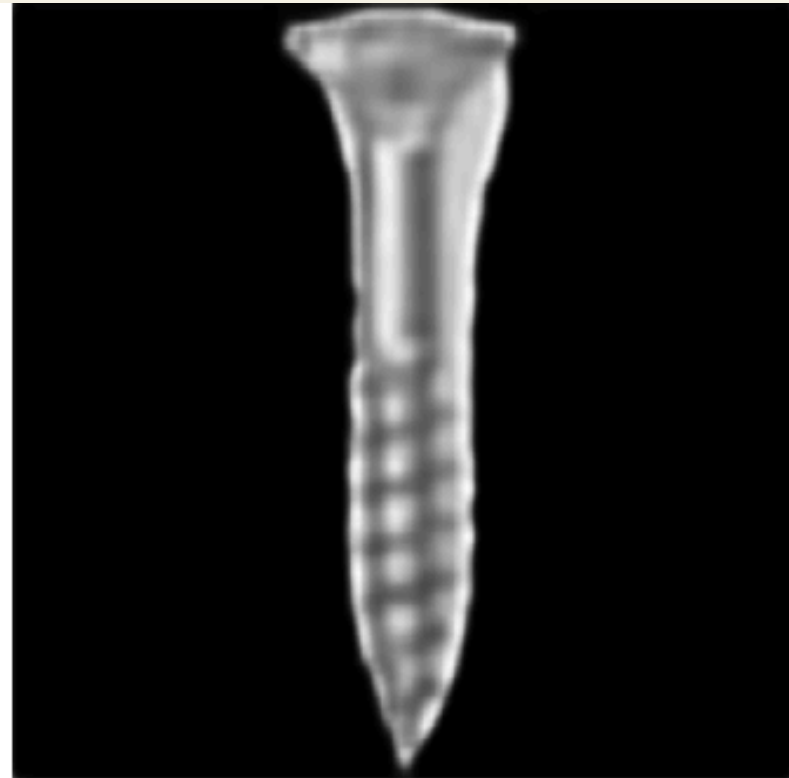
Autoencoders

MSE-V1 + DA



Autoencoders

MSE-V1 + DA



Autoencoders

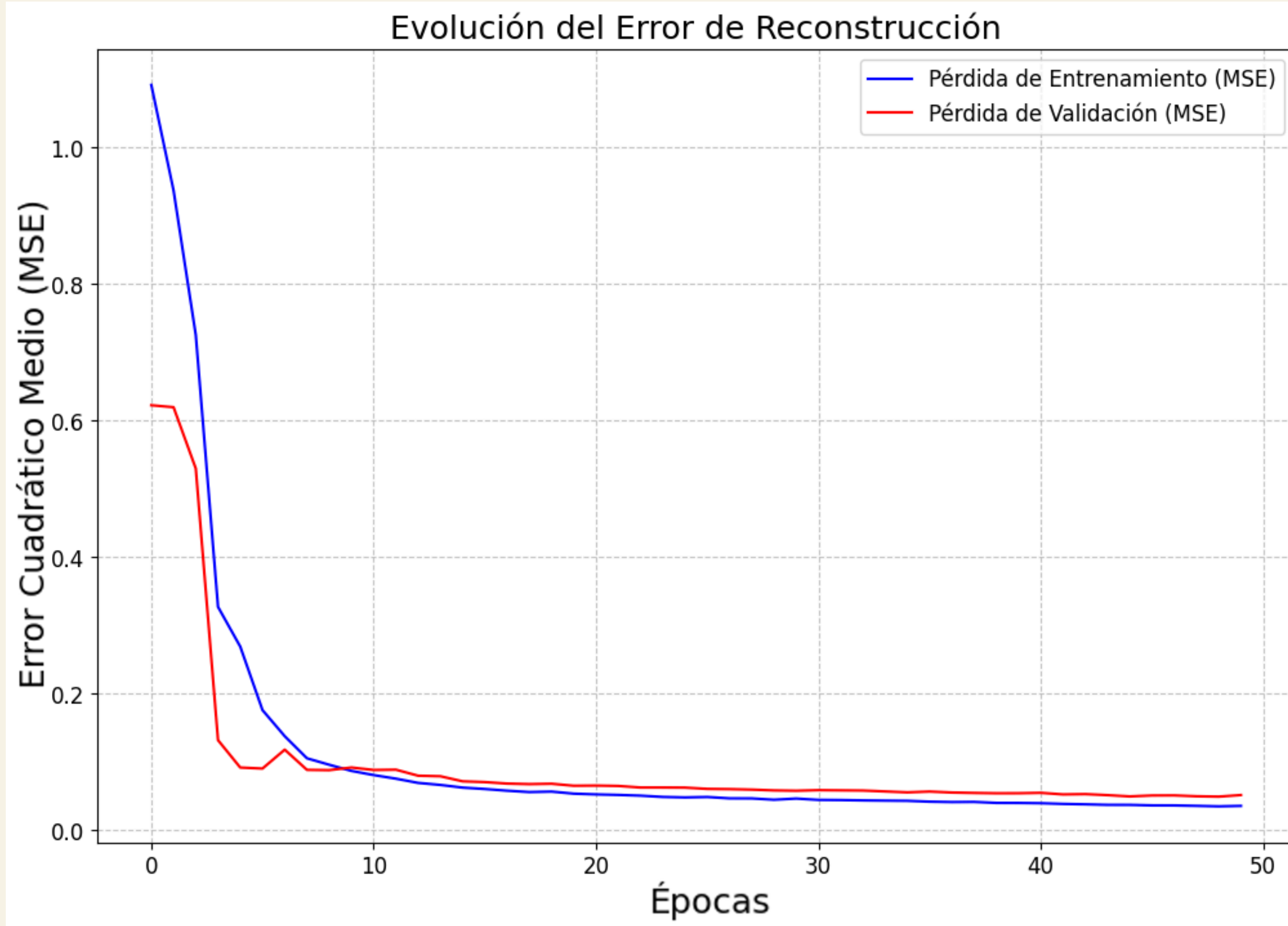
El Mediano

Metricas			
Versiones	Accuracy	Recall	F1-score
DA	0.28	N:1 A:0	N:0.44 A:0
MSE-V2 + DA	0.28	N:1 A:0	N:0.44 A:0

MSE-V2: Peso del fondo 1, peso de contorno: 8
mas SSIM (SSIM_LOSS=1-SSIM)

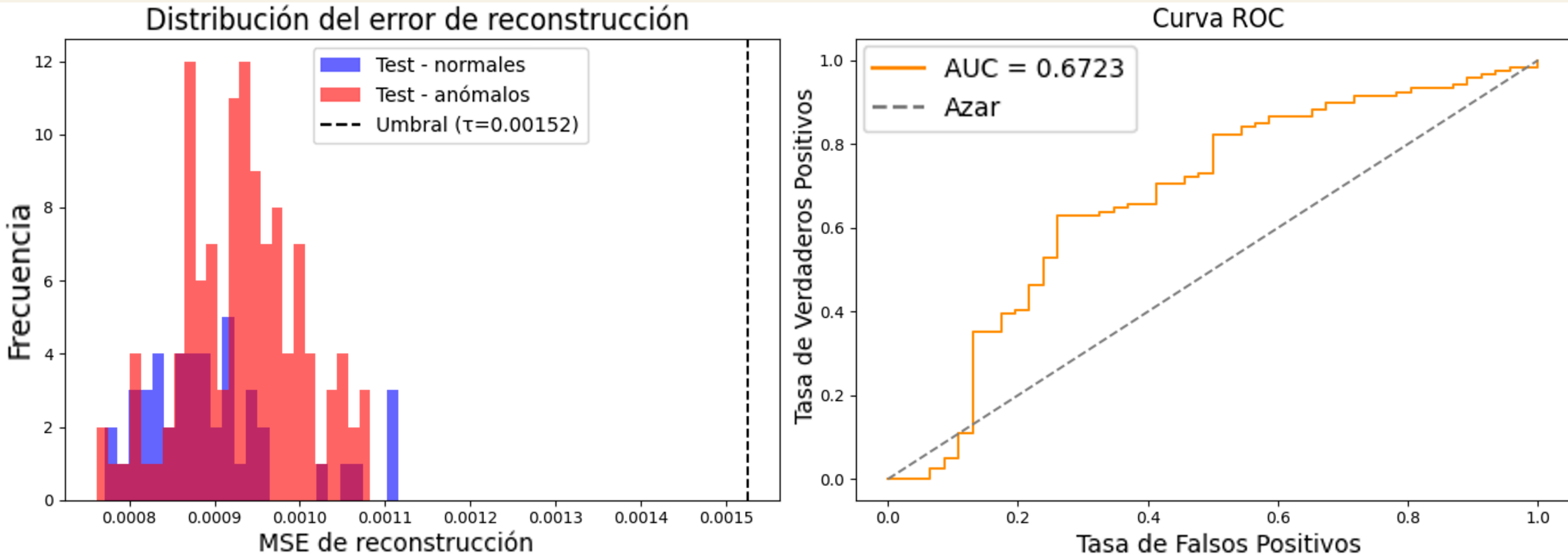
Autoencoders

El Mediano



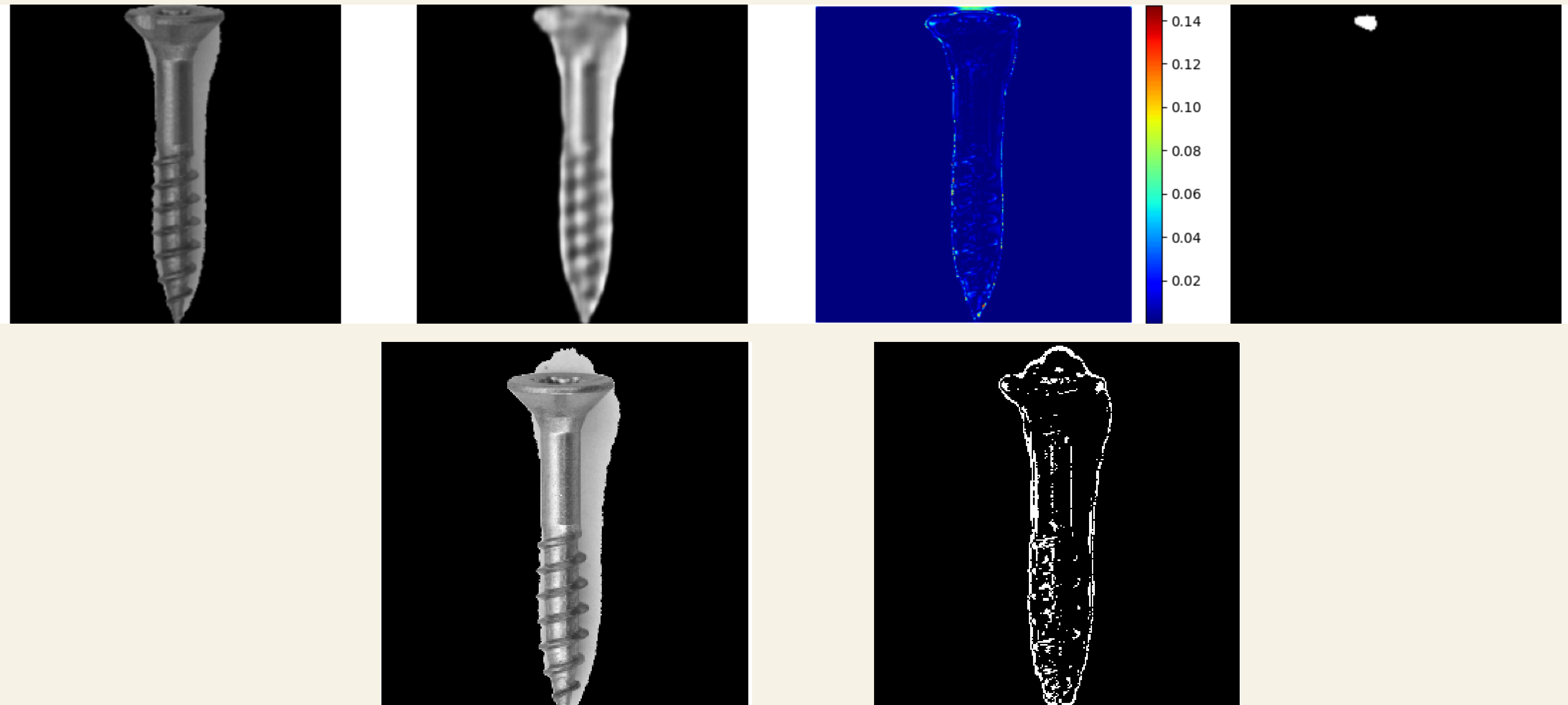
Autoencoders

El Mediano



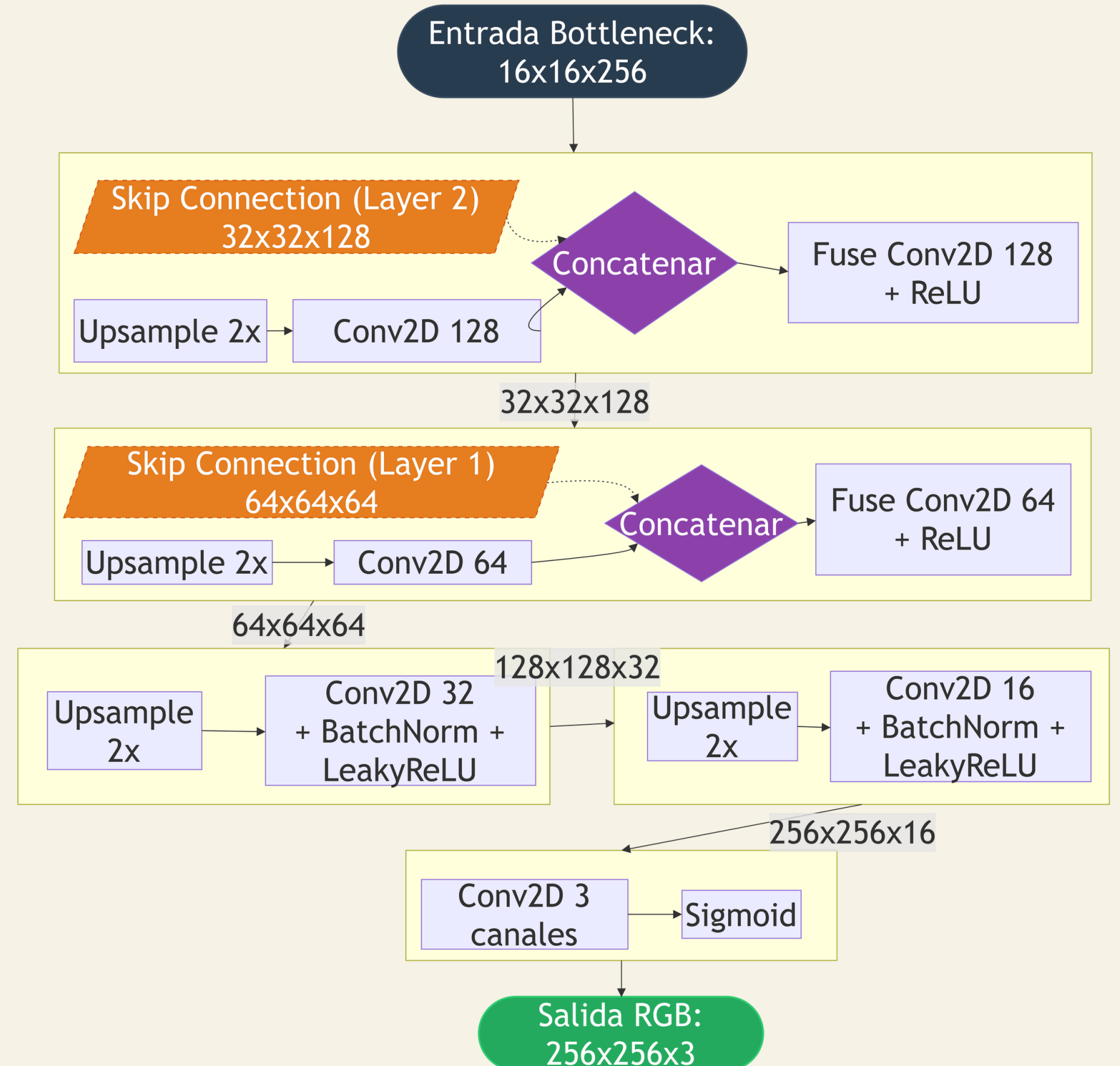
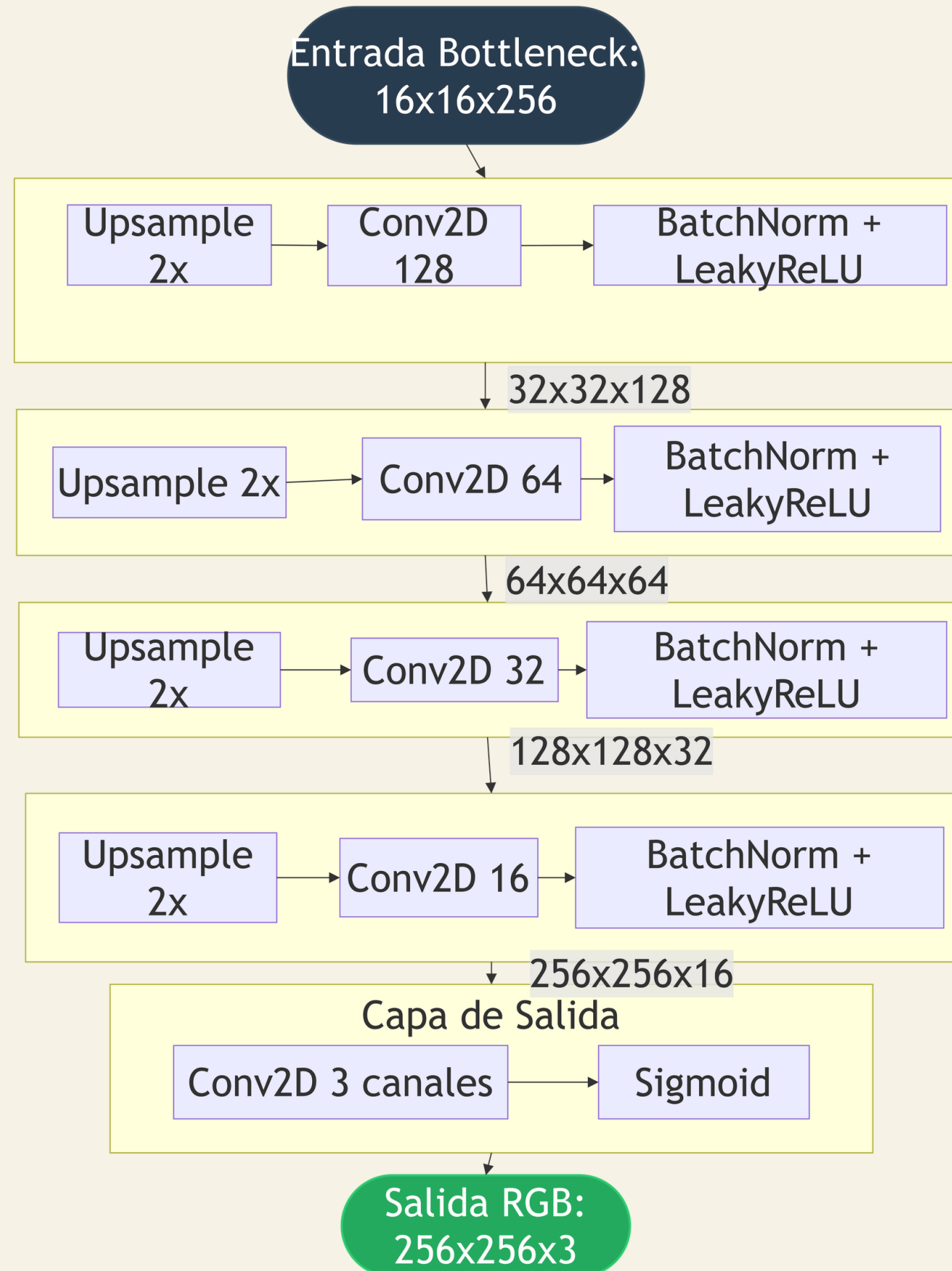
Autoencoders

El Mediano



Transfer Learning

Backbone: ResNet18 – Layer3 16x16x256



Transfer Learning

Funciones de pérdida

V1

MSE_LOSS: Donde se tiene mse, pero con el fondo con un peso (1) y el polígono mínimo de tornillo en otro (8).

V2

MAE_LOSS: MAE con el mismo sistema de pesos.

V3

MAE_Y_SSIM_LOSS: MAE más SSIM donde al final se suman de la siguiente manera

$$e = A * mae + (1 - A) * SSIM \quad A \text{ entre } 0 \text{ y } 1$$

V4

MSE_Y_SSIM_LOSS: MAE más SSIM donde al final se suman de la siguiente manera

$$e = A * mse + (1 - A) * SSIM \quad A \text{ entre } 0 \text{ y } 1$$

Optimizador: Adam, Lr= 5e-4, Batch size=16, A: 0.5, Peso de tornillo:20 y peso fondo=1

Transfer Learning

Metricas			
Versiones	Accuracy	Recall	F1-score
V1	0.63	N:0.61 A:0.64	N:0.48 A:0.71
V2	0.54	N:0.67 A:0.49	N:0.45 A:0.60
V3	0.55	N:0.67 A:0.50	N:0.46 A:0.62
V3+DA	0.43	N:0.80 A:0.29	N:0.44 A:0.42
V4	0.68	N:0.48 A:0.76	N:0.46 A:0.78
V4+DA	0.53	N:0.63 A:0.49	N:0.43 A:0.60

Transfer Learning

Métricas

Evaluando 165 imágenes de test...

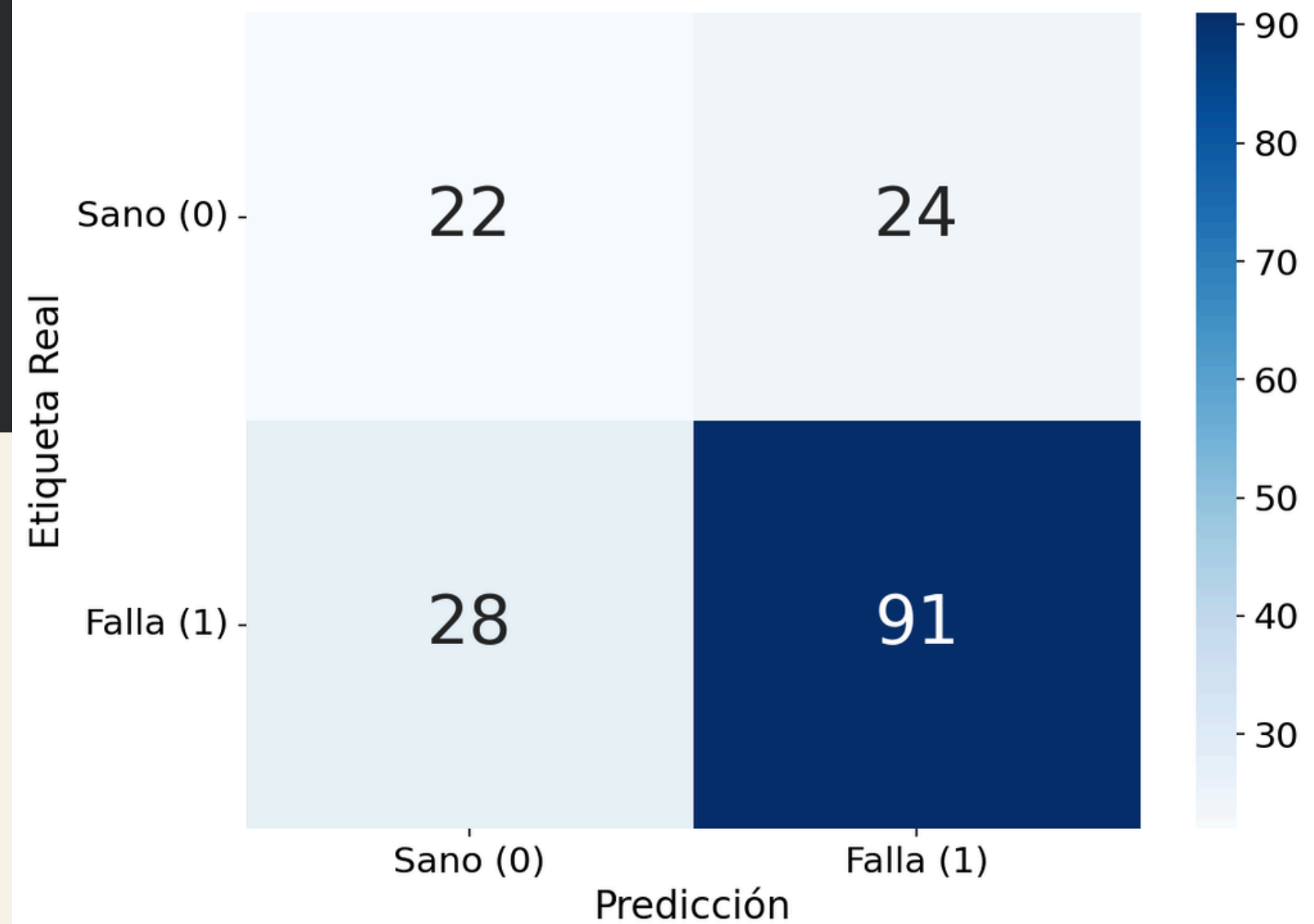
--- REPORTE FINAL EN TEST (Umbral: 0.002387) ---

	precision	recall	f1-score	support
Sano	0.44	0.48	0.46	46
Falla	0.79	0.76	0.78	119
accuracy			0.68	165
macro avg	0.62	0.62	0.62	165
weighted avg	0.69	0.68	0.69	165

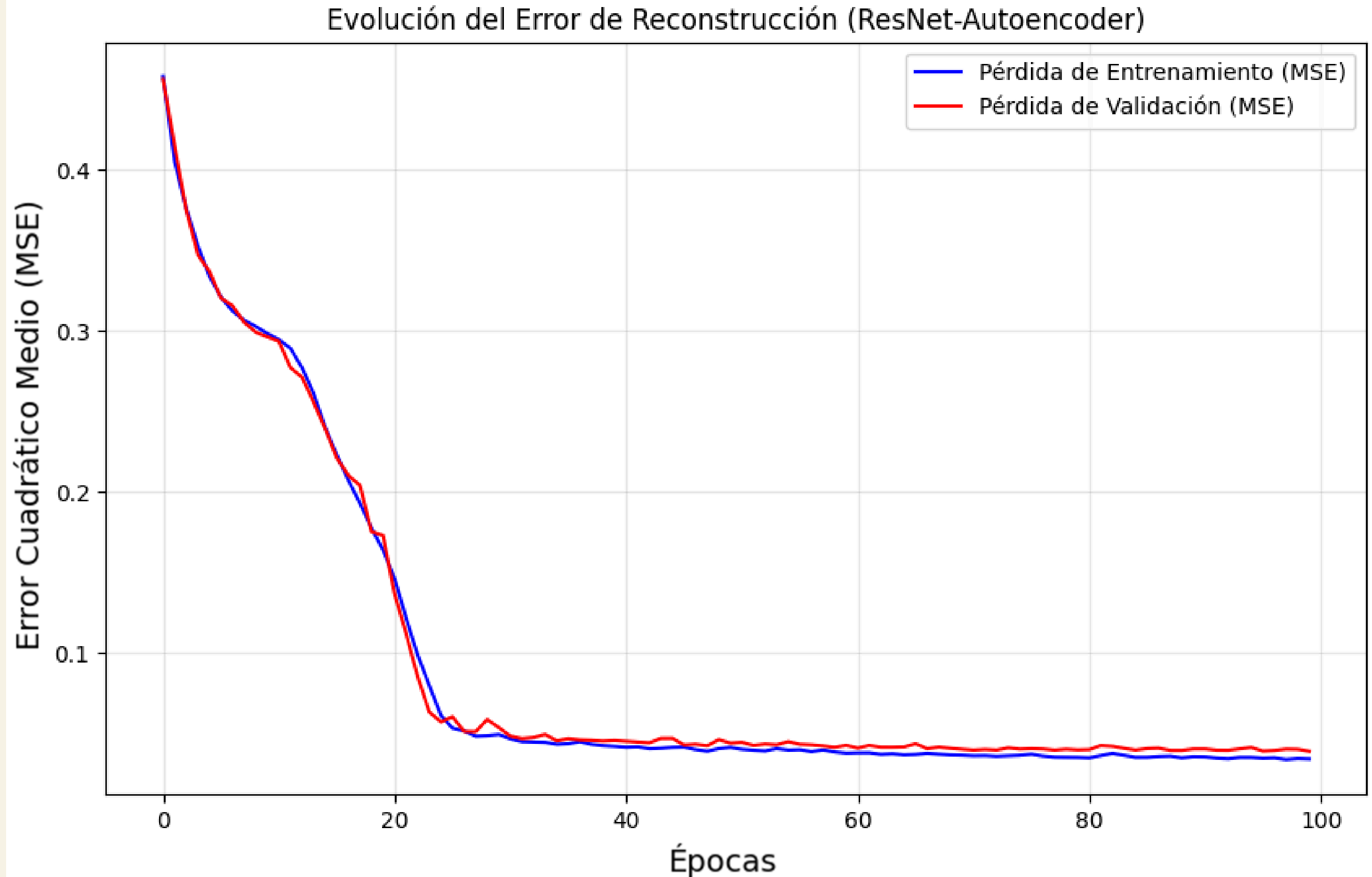
AUC-ROC: 0.6461

Matriz de confusión

Matriz de Confusión (ResNet-Autoencoder)



Transfer Learning



Transfer Learning

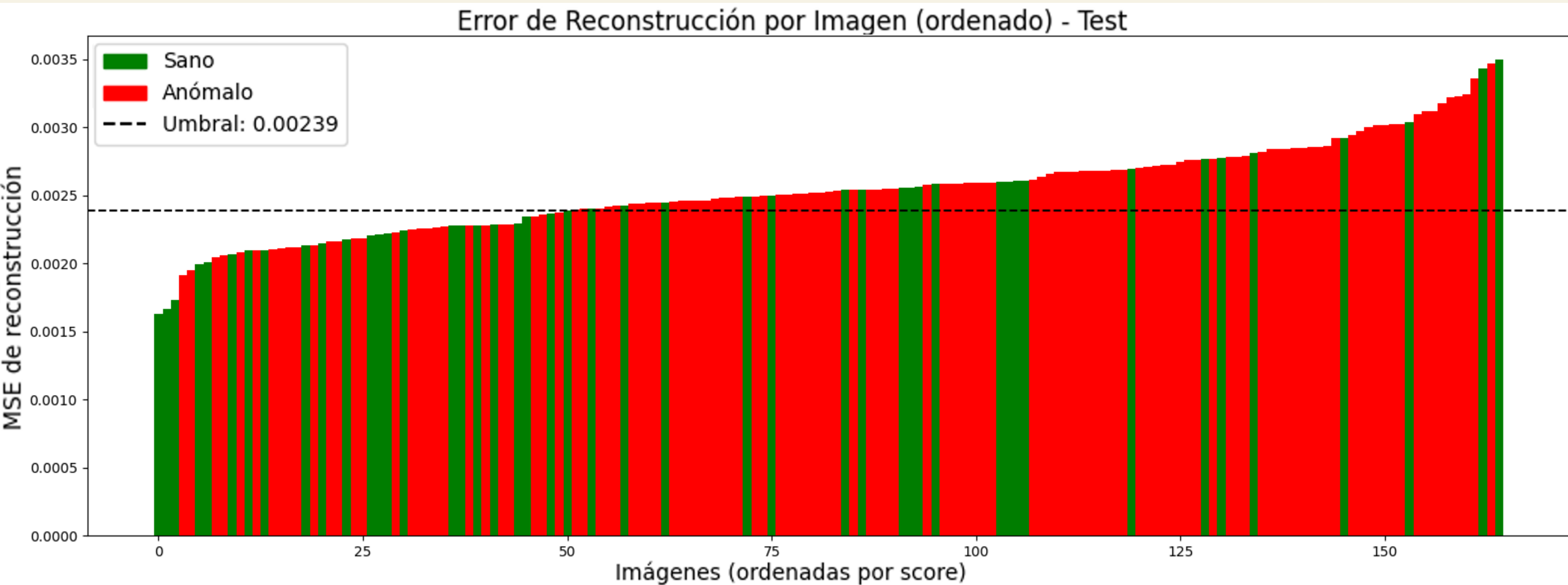
Reconstrucciones
Tornillo sano



Tornillo con falla

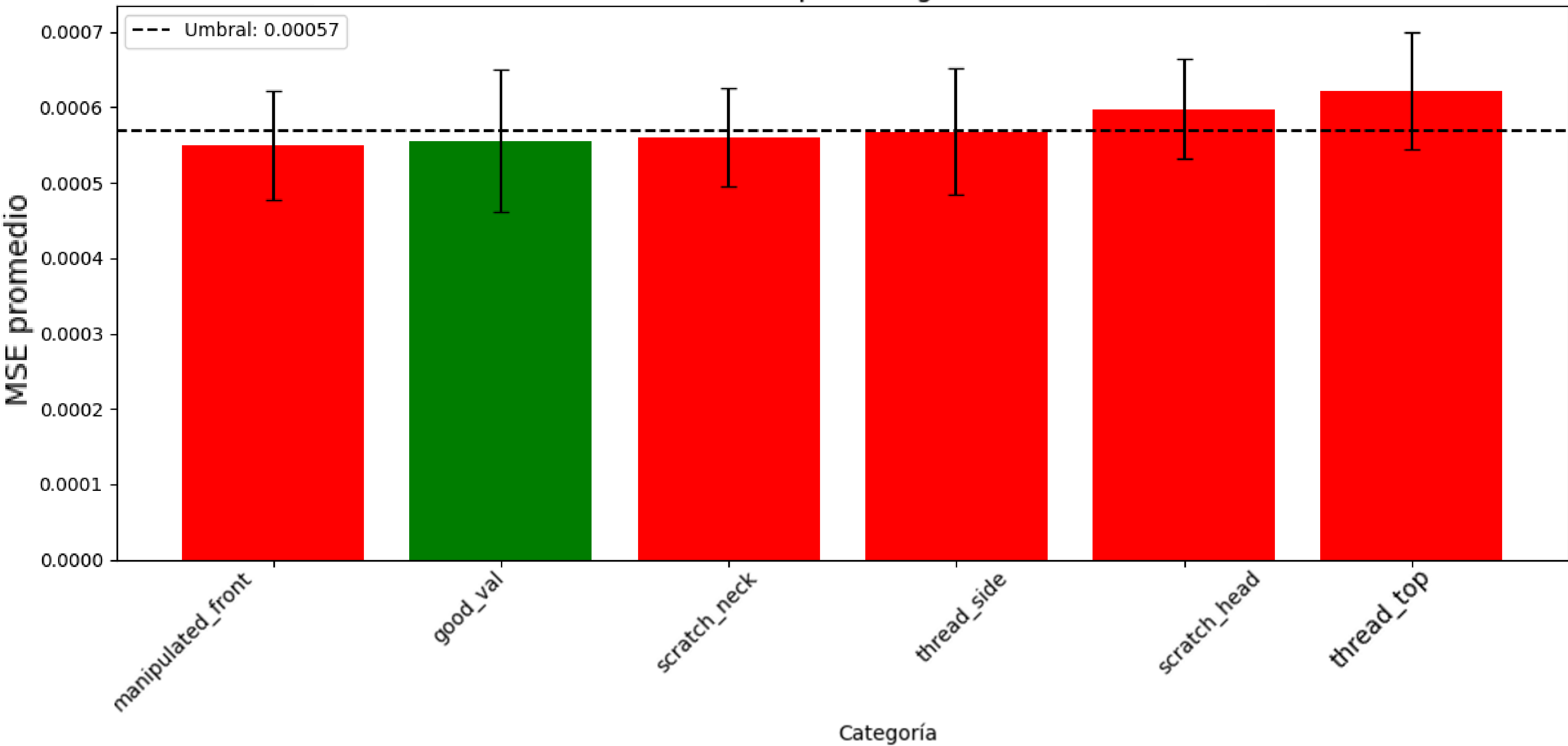


Transfer Learning



Transfer Learning

Error de Reconstrucción Promedio por Categoría ($\pm$ desvío estándar)

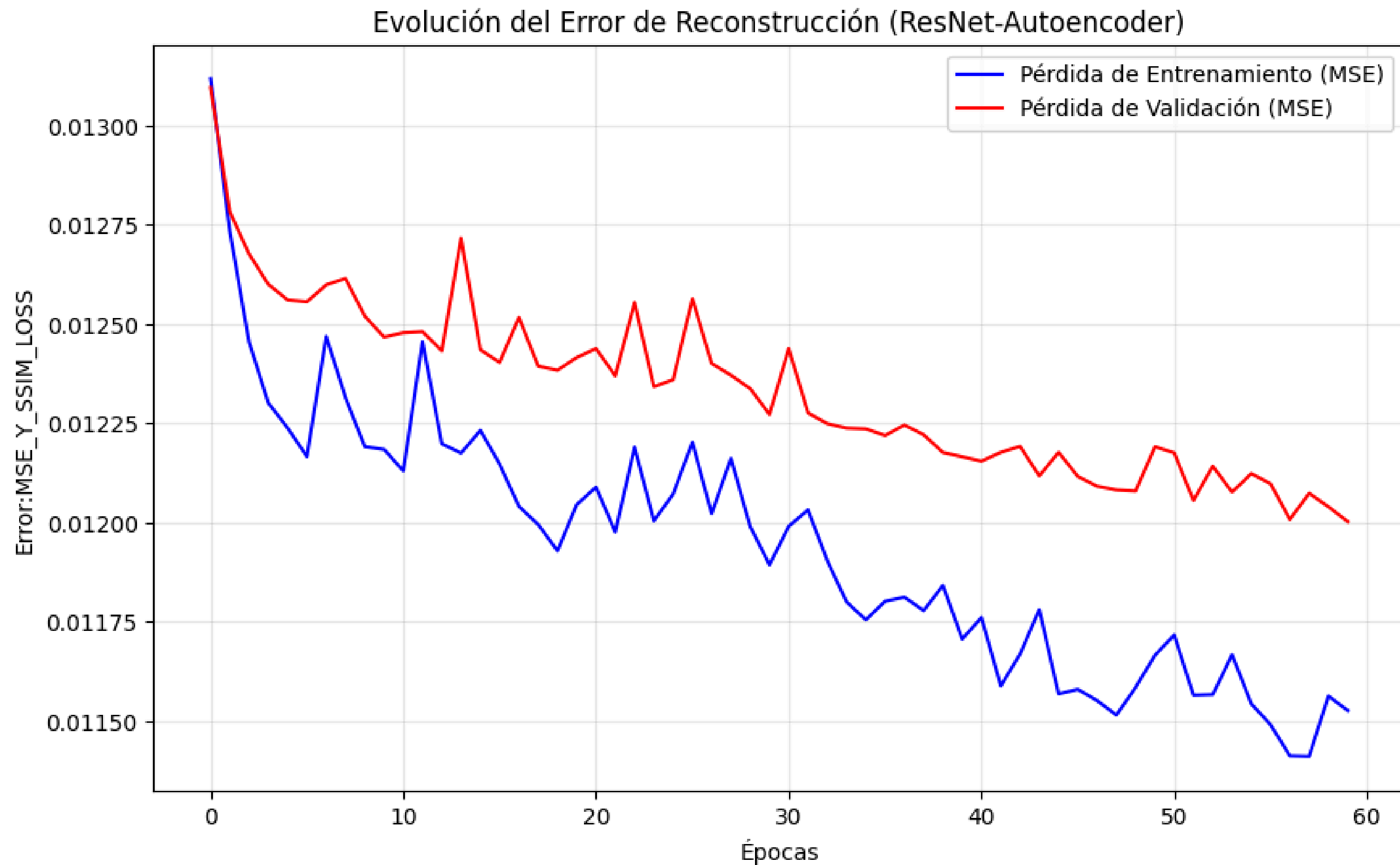


Fine Tuning

Para este paso Se descongela el layer 3

- Para el Encoder layer3 $lr=1e-5$
- Para el Decoder $lr=5e-5$
- Épocas=60
- Paciencia=6, Mínimo cambio= $1e-5$

Fine Tuning



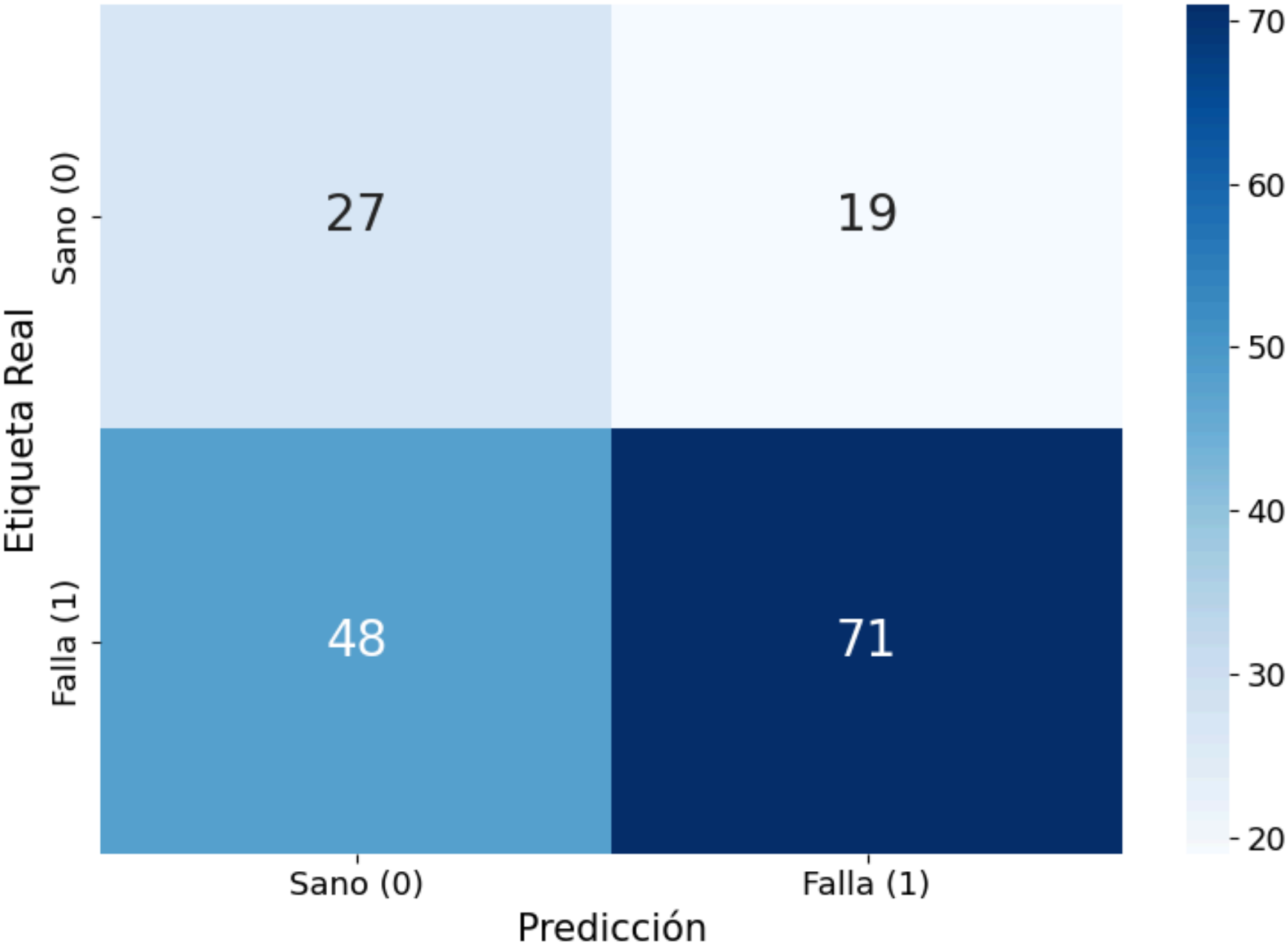
Fine Tuning

Métricas

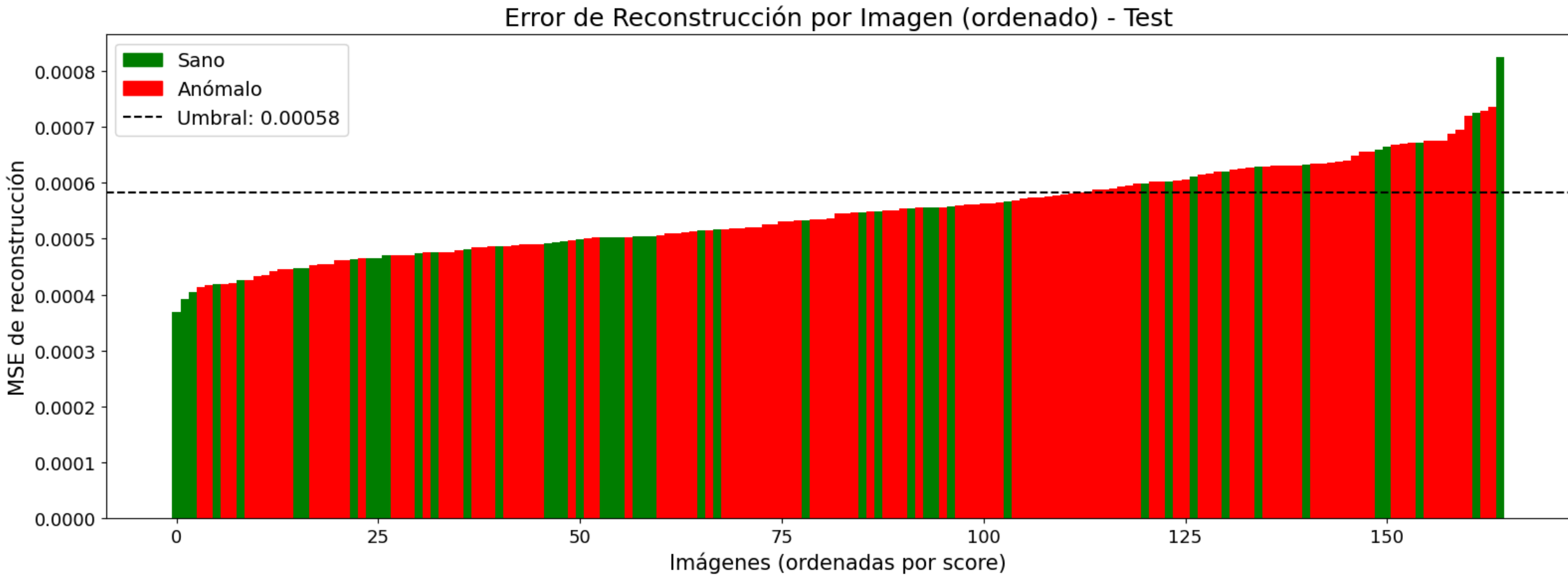
--- REPORTE FINAL EN TEST (Umbral: 0.000521) ---				
	precision	recall	f1-score	support
Sano	0.36	0.59	0.45	46
Falla	0.79	0.60	0.68	119
accuracy			0.59	165
macro avg	0.57	0.59	0.56	165
weighted avg	0.67	0.59	0.61	165
AUC-ROC: 0.5881				

Matriz de confusión

Matriz de Confusión - Evaluación Final (ResNet-Autoencoder)

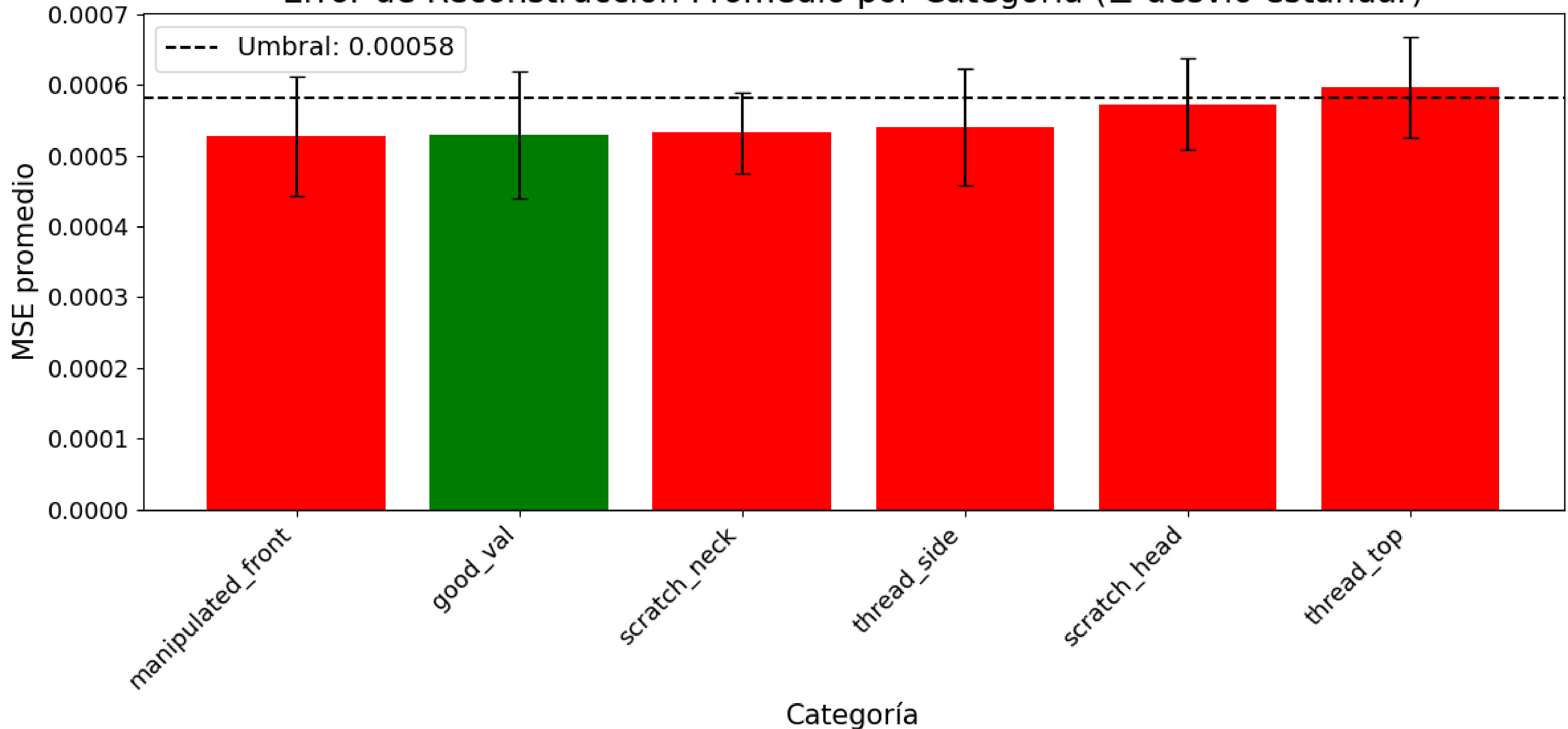


Fine Tuning



Fine Tuning

Error de Reconstrucción Promedio por Categoría ($\pm$ desvío estándar)



Patchcore

¿Cómo funciona?

1. Extracción de características.

a. Extrae características con Wide Resnet50-2.

2. Creación del banco de memoria.

a. Toma los tensores del Layer 2 y Layer 3, y los unifica.

3. Reducción de la cantidad de parches.

a. Se toma un 10% de los parches eligiendo mediante un algoritmo K-Center Greedy.

4. Calibración del Umbral .

a. Calcula un umbral mediante muestras de validación.

5. Inferencia y Detección.

a. Compara los parches de los tornillos nuevos con los 9 vecinos más cercanos, y reescala los resultados.

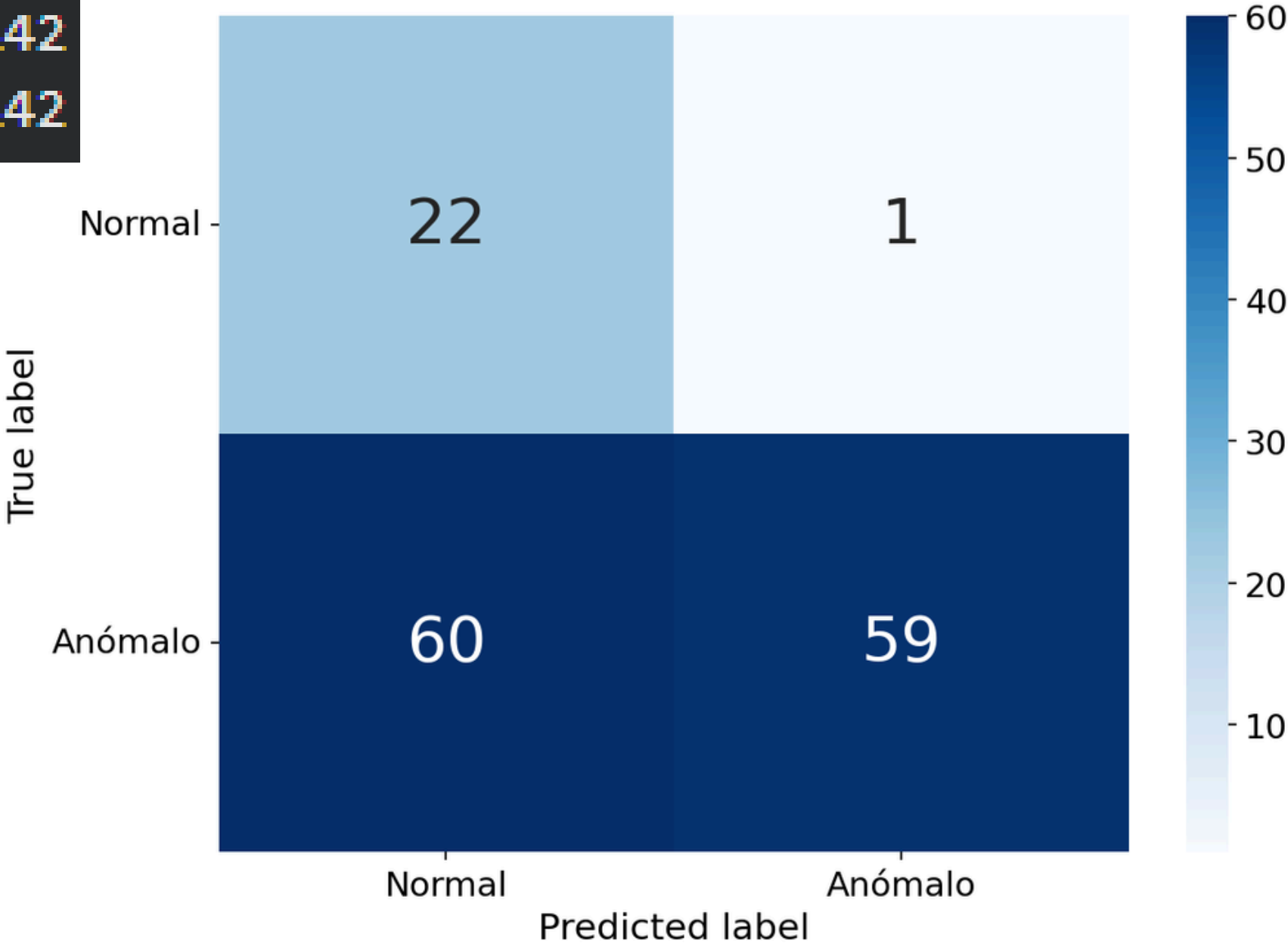
Patchcore

Métricas

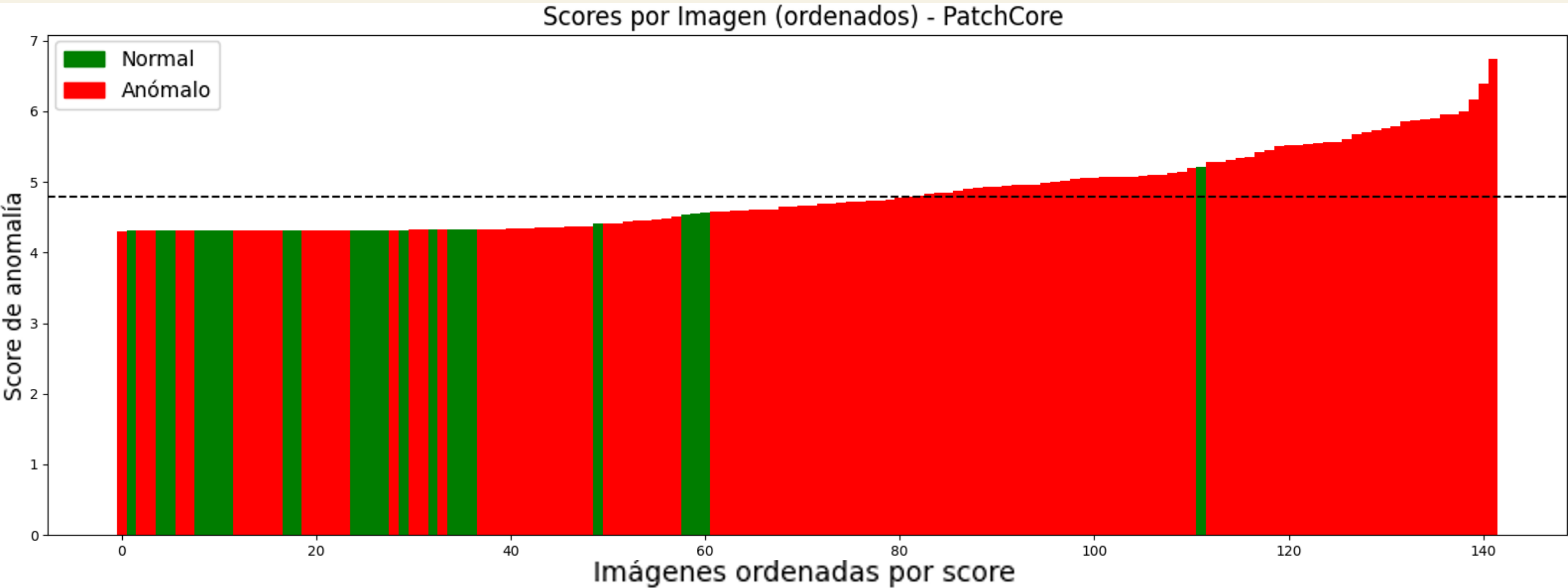
	precision	recall	f1-score	support
Normal	0.27	0.96	0.42	23
Anómalo	0.98	0.50	0.66	119
accuracy			0.57	142
macro avg	0.63	0.73	0.54	142
weighted avg	0.87	0.57	0.62	142

Matriz de confusión

Matriz de Confusión - PatchCore (umbral=4.788)



Patchcore



Patchcore

Resultados por clase

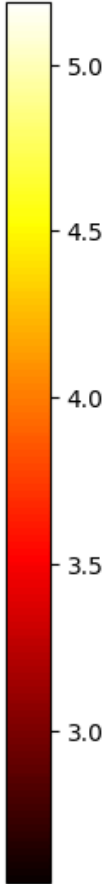
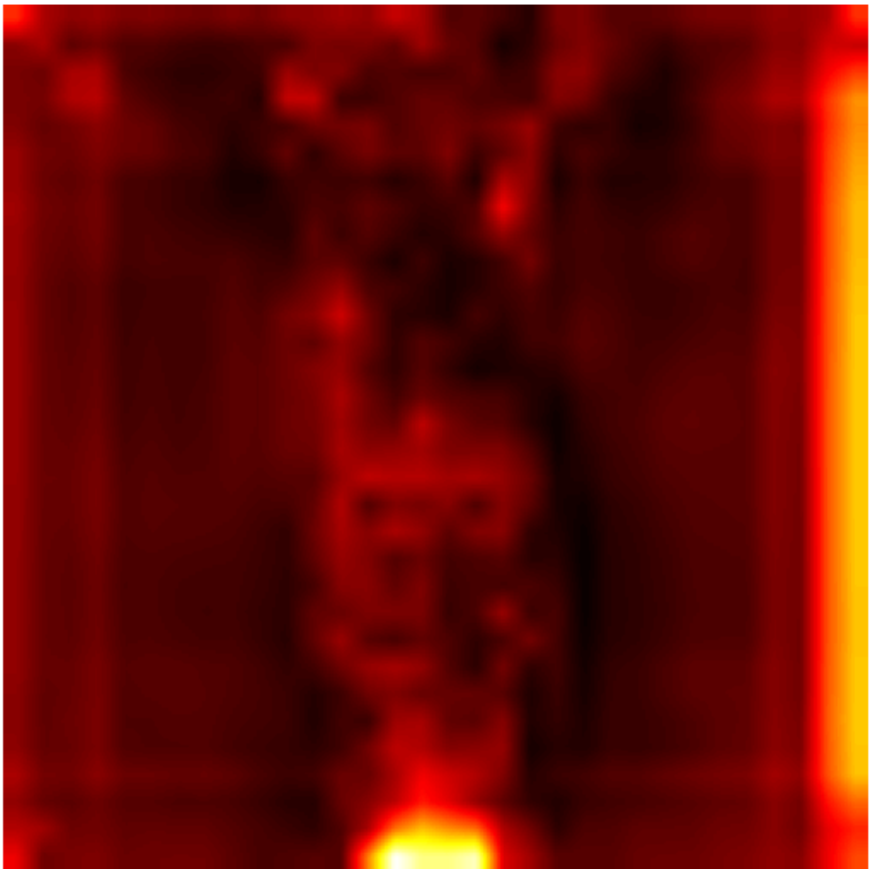
categoria	n_imagenes	detectados	tasa_deteccion	score_promedio
manipulated_front	24.0	22.0	0.916667	5.432585
scratch_head	24.0	10.0	0.416667	4.754149
scratch_neck	25.0	16.0	0.640000	5.090587
thread_side	23.0	5.0	0.217391	4.597372
thread_top	23.0	6.0	0.260870	4.630310

Sano con mayor score

Sano con MAYOR score
Score: 5.2071

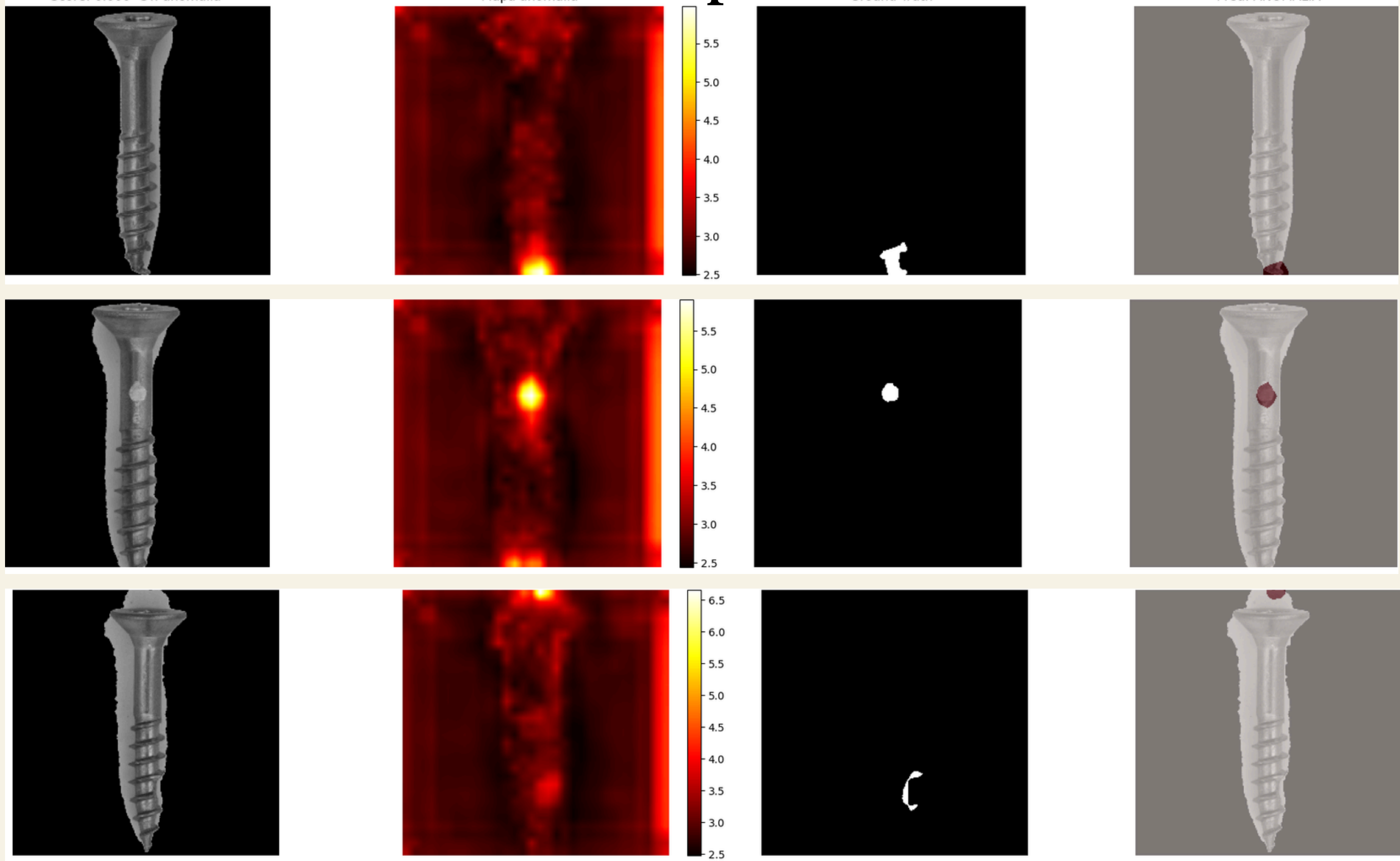


Mapa de anomalía



Patchcore

Máscaras predichas



Conclusión

1. Dataset limitado.
2. Generalización.
3. Sesgos.
4. Implementación industrial.



Futuras implementaciones

1. Ajustar la función de alineación y eliminación de fondo.
2. Realizar un Dataset casero con el cual comprobar los distintos modelos.
3. Probar diferentes extractores de características.
4. Optimizar el pipeline para obtener resultados que sean implementables en la industria.

¿Preguntas?

No se puede conectar con el backend de GPU

En estos momentos no puedes conectarte a una GPU debido a los límites de uso de Colab. [Más información](#)

Para tener un mayor acceso a GPUs, puedes comprar unidades de computación de Colab con [Pay As You Go](#).

Cerrar

[Conectarse sin GPU](#)